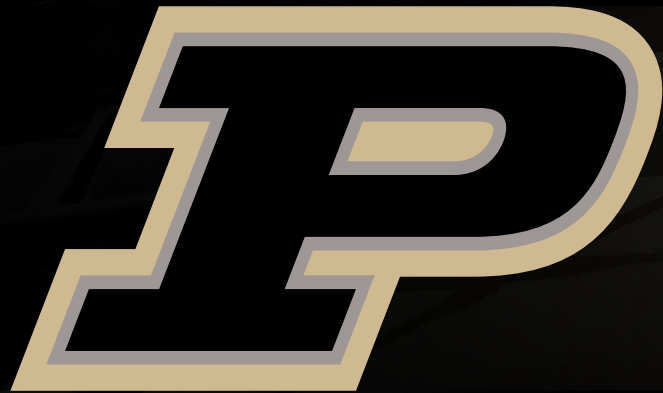




Purdue Waffle Wizards

Wilbert Chu
Robert Lee
Thomas Marlowe

ICPC WF 2025
Team Reference Document



- 1 Contest
- 2 Data structures
- 3 Numerical
- 4 Number theory
- 5 Graph
- 6 Geometry
- 7 Strings
- 8 Math
- 9 Sources

Contest (1)

template.cpp

14 lines

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

.bashrc

3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
-fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <>
```

.vimrc

5 lines

```
set cin ai is ts=2 sw=2 tm=50 nu bg=dark ru bo=all mouse=a | sy
on
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \ | tr -d '[:space:]' \
\ | md5sum \ | cut -c-6
```

hash.sh

3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |cut -c-6
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.
```

```
Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
```

```
1 Read the full problem statement again.
1 Do you handle all corner cases correctly?
Have you understood the problem correctly?
3 Any uninitialized variables?
Any overflows?
6 Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
8 What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
13 Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
18 Go through the algorithm for a simple case.
Go through this list again.
20 Explain your algorithm to a teammate.
Ask the teammate to look at your code.
24 Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
```

```
Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).
```

```
Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?
```

```
Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Random.h

Description: RNG in C++ for Wilbert.

6741a4, 12 lines

```
1mt19937 rng(time(0)); // 32 bits
2mt19937_64 rng64(time(0)); // 64 bits
3rng(), rng64(); // random unsigned 32 bit and 64 bit resp.
4uniform_int_distribution<>(0,10)(rng); // random integer in
↪ [0,10]
5uniform_real_distribution<>(0,10)(rng); // random double in
↪ [0,10]
6normal_distribution<>(0,10)(rng); // mean=0, stddev=10
7// like every other distribution exists too, look in offline C++
↪ reference.
8vector<int> vec = {0,1,2};
9ranges::shuffle(vec, rng); // shuffle vector
10vector<int> sample;
11ranges::sample(vec, back_inserter(sample), 2, rng); // sample 2
↪ elements from vec, push to sample
12minstd_rand min_rng; // much faster RNG with limited randomness
```

1.1 Pragmas

- **#pragma** GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- **#pragma** GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- **#pragma** GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

Data structures (2)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type.

Time: $\mathcal{O}(\log N)$

782797, 14 lines

```
1#include <bits/extc++.h>
2using namespace __gnu_pbds;
3
4template<class T>
5using Tree = tree<T, null_type, less<T>, rb_tree_tag,
6    tree_order_statistics_node_update>;
7
8void example() {
9    Tree<int> t, t2; t.insert(8);
10    auto it = t.insert(10).first;
11    assert(it==t.lower_bound(9)); assert(t.order_of_key(10)==1);
12    assert(t.order_of_key(11)==2); assert(*t.find_by_order(0)==8);
13    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
14}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~ 3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
1#include <bits/extc++.h>
2// To use most bits rather than just the lowest ones:
3struct chash { // large odd number for C
4    const uint64_t C = ll(4e18 * acos(0)) | 71;
5    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
6};
7__gnu_pbds::gp_hash_table<ll,int,chash> h({}, {}, {}, {}, {1<<16});
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st.time() and rollback().

Usage: int t = uf.time(); ...; uf.rollback(t);

Time: $\mathcal{O}(\log(N))$

de4ad0, 22 lines

```
1struct RollbackUF {
2    vi e; vector<pii> st; // remove st for no rollback
3    RollbackUF(int n) : e(n, -1) {}
4    int size(int x) { return -e[find(x)]; }
5    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
6    // int find(int x) { return e[x] < 0 ? x : e[x]=find(e[x]); }
7    int time() { return sz(st); } //
8    void rollback(int t) { //
9        for (int i = time(); i --> t;)
10            e[st[i].first] = st[i].second;
11        st.resize(t);
12    }
13    bool join(int a, int b) {
14        a = find(a), b = find(b);
15        if (a == b) return false;
16        if (e[a] > e[b]) swap(a, b);
17        st.push_back({a, e[a]}); //
18        st.push_back({b, e[b]}); //
19        e[a] += e[b]; e[b] = a;
20        return true;
21    }
22};
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

0f4bdb, 19 lines

```
1struct Tree {
2    typedef int T;
3    static constexpr T unit = INT_MIN;
4    T f(T a, T b) { return max(a, b); } // (any associative fn)
5    vector<T> s; int n;
6    Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
```

```
7 void update(int pos, T val) {
8     for (s[pos += n] = val; pos /= 2;)
9         s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
10 }
11 T query(int b, int e) { // query [b, e)
12     T ra = unit, rb = unit;
13     for (b += n, e += n; b < e; b /= 2, e /= 2) {
14         if (b % 2) ra = f(ra, s[b++]);
15         if (e % 2) rb = f(s[--e], rb);
16     }
17     return f(ra, rb);
18 }
19};
```

LazySegmentTree.h

Description: TAG (a,b) means apply $f(x) = ax+b$ to all elements in a range. Segments contain either MAX or +, and you're able to recompute a segment if all elements have a TAG applied to it.

Usage: On each test case, reset the value of N. Initialize seg (and len) from $i = N$ to $2N$, and iterate backwards from $N-1$ to 0 to build seg and lazy

17f69b, 76 lines

```
1 const int MAXN = 200000;
2 struct Lazy{
3     using T = long long;
4     using TAG = pair<T,T>;
5     static constexpr T defT = 0; //<- edit
6     static constexpr TAG defTAG = {1,0}; //<- edit
7
8     int N;
9     TAG lazy[MAXN];
10    T seg[2*MAXN];
11    int len[2*MAXN];
12
13    T f( T a, T b ){ return a+b; } //merge two segments
14    void apply( TAG f, int i ){
15        TAG g = lazy[i];
16        if(i < N) lazy[i] = { f.first*g.first, f.first*g.second +
17                               ↪ f.second }; //compose two tags
18        seg[i] = f.first*seg[i] + f.second*len[i]; //evaluate tag on
19                               ↪ segment
20    }
21
22    void hammer_down( int i ){
23        for( int k = 20; k > 0; k-- ){
24            int j = (i>>k);
25            if(j==0) continue;
26            apply( lazy[j], 2*j );
27            apply( lazy[j], 2*j+1 );
28            lazy[j] = defTAG;
29        }
30    }
31
32    void boiler_up( int i ){
33        i /= 2;
34        while(i){
35            seg[i] = f(seg[2*i],seg[2*i+1]);
36            i /= 2;
37        }
38    }
39
40    void update( int l, int r, TAG tag ){
41        l += N, r += N;
42        int l0 = l, r0 = r;
43        while(l0%2==0) l0 /= 2;
44        while(r0%2==0) r0 /= 2;
45        hammer_down(l0);
46        hammer_down(r0-1);
47        while( l < r ){
48            if(l&1) apply(tag,l++);
49            if(r&1) apply(tag,--r);
50            l /= 2, r /= 2;
51        }
52        boiler_up(l0);
53        boiler_up(r0-1);
54    }
55
56    T query( int l, int r ){ //range sum on [l,r)
57        l += N, r += N;
58        hammer_down(l);
```

```
54 hammer_down(r-1);
55 T m1 = defT, m2 = defT;
56 while( l < r ){
57     if(l&1) m1 = f(m1,seg[l++]);
58     if(r&1) m2 = f(seg[--r],m2);
59     l /= 2, r /= 2;
60 }
61 return f(m1,m2);
62 }
63
64 void build( int n, vector<T> &a ){
65     N = n;
66     for( int i = 0; i < n; i++ ){
67         seg[i+n] = a[i];
68         len[i+n] = 1;
69     }
70     for( int i = n-1; i > 0; i-- ){
71         lazy[i] = defTAG;
72         seg[i] = f(seg[2*i], seg[2*i+1]);
73         len[i] = len[2*i] + len[2*i+1];
74     }
75 }
76};
```

KactlLazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.

Usage: $\text{Node} * \text{tr} = \text{new Node}(v, 0, \text{sz}(v))$;

Time: $\mathcal{O}(\log N)$.

34ecf5, 50 lines

```
1 const int inf = 1e9;
2 struct Node {
3     Node *l = 0, *r = 0;
4     int lo, hi, mset = inf, madd = 0, val = -inf;
5     Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
6     Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
7         if (lo + 1 < hi) {
8             int mid = lo + (hi - lo)/2;
9             l = new Node(v, lo, mid); r = new Node(v, mid, hi);
10            val = max(l->val, r->val);
11        }
12        else val = v[lo];
13    }
14    int query(int L, int R) {
15        if (R <= lo || hi <= L) return -inf;
16        if (L <= lo && hi <= R) return val;
17        push();
18        return max(l->query(L, R), r->query(L, R));
19    }
20    void set(int L, int R, int x) {
21        if (R <= lo || hi <= L) return;
22        if (L <= lo && hi <= R) mset = val = x, madd = 0;
23        else {
24            push(), l->set(L, R, x), r->set(L, R, x);
25            val = max(l->val, r->val);
26        }
27    }
28    void add(int L, int R, int x) {
29        if (R <= lo || hi <= L) return;
30        if (L <= lo && hi <= R) mset = val = x, madd += x;
31        else madd += x;
32        val += x;
33    }
34    }
35    else {
36        push(), l->add(L, R, x), r->add(L, R, x);
37        val = max(l->val, r->val);
38    }
39    }
40    void push() {
41        if (!l) {
42            int mid = lo + (hi - lo)/2;
43            l = new Node(lo, mid); r = new Node(mid, hi);
44        }
45        if (mset != inf)
46            l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
```

```
47     else if (madd)
48         l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
49 }
50};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).

Usage: $\text{SubMatrix}<\text{int}> \text{m}(\text{matrix})$;

$\text{m.sum}(0, 0, 2, 2)$; // top left 4 elements

Time: $\mathcal{O}(N^2 + Q)$

c59ada, 13 lines

```
1 template<class T>
2 struct SubMatrix {
3     vector<vector<T>>> p;
4     SubMatrix(vector<vector<T>>& v) {
5         int R = sz(v), C = sz(v[0]);
6         p.assign(R+1, vector<T>(C+1));
7         rep(r,0,R) rep(c,0,C)
8             p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
9     }
10    T sum(int u, int l, int d, int r) {
11        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
12    }
13};
```

Matrix.h

Description: Basic operations on square matrices.

Usage: $\text{Matrix}<\text{int}, 3> \text{A}$;

$\text{A.d} = \{\{\{1,2,3\}\}, \{\{4,5,6\}\}, \{\{7,8,9\}\}\}$;

$\text{array}<\text{int}, 3> \text{vec} = \{1,2,3\}$;

$\text{vec} = (\text{A}^N) * \text{vec}$;

6ab5db, 26 lines

```
1 template<class T, int N> struct Matrix {
2     typedef Matrix M;
3     array<array<T, N>, N> d{};
4     M operator*(const M& m) const {
5         M a;
6         rep(i,0,N) rep(j,0,N)
7             rep(k,0,N) a.d[i][j] += d[i][k]*m.d[k][j];
8         return a;
9     }
10    array<T, N> operator*(const array<T, N>& vec) const {
11        array<T, N> ret{};
12        rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
13        return ret;
14    }
15    M operator^(ll p) const {
16        assert(p >= 0);
17        M a, b(*this);
18        rep(i,0,N) a.d[i][i] = 1;
19        while (p) {
20            if (p&1) a = a*b;
21            b = b*b;
22            p >>= 1;
23        }
24        return a;
25    }
26};
```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).

Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```
1 struct Line {
2     mutable ll k, m, p;
3     bool operator<(const Line& o) const { return k < o.k; }
4     bool operator<(ll x) const { return p < x; }
5 };
6
7 struct LineContainer : multiset<Line, less<>> {
8     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
9     static const ll inf = LLONG_MAX;
```



```
10 ll div(ll a, ll b) { // floored division
11     return a / b - ((a ^ b) < 0 && a % b); }
12 bool isect(iterator x, iterator y) {
13     if (y == end()) return x->p = inf, 0;
14     if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
15     else x->p = div(y->m - x->m, x->k - y->k);
16     return x->p >= y->p;
17 }
18 void add(ll k, ll m) {
19     auto z = insert({k, m, 0}), y = z++, x = y;
20     while (isect(y, z)) z = erase(z);
21     if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
22     while ((y = x) != begin() && (--x)->p >= y->p)
23         isect(x, erase(y));
24 }
25 ll query(ll x) {
26     assert(!empty());
27     auto l = *lower_bound(x);
28     return l.k * x + l.m;
29 }
30};
```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$

1048b8, 42 lines

```
1 struct Node {
2     Node *l = 0, *r = 0; int val, y, c = 1;
3     Node(int val) : val(val), y(rand()) {}
4     void recalc();
5 };
6 int cnt(Node* n) { return n ? n->c : 0; }
7 void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
8
9 template<class F> void each(Node* n, F f) {
10     if (n) { each(n->l, f); f(n->val); each(n->r, f); }
11 }
12 pair<Node*, Node*> split(Node* n, int k) {
13     if (!n) return {};
14     if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
15         auto pa = split(n->l, k);
16         n->l = pa.second; n->recalc();
17         return {pa.first, n};
18     } else {
19         auto pa = split(n->r, k - cnt(n->l) - 1); // and just "k"
20         n->r = pa.first; n->recalc();
21         return {n, pa.second};
22     }
23 }
24 Node* merge(Node* l, Node* r) {
25     if (!l) return r; if (!r) return l;
26     if (l->y > r->y) {
27         l->r = merge(l->r, r); l->recalc(); return l;
28     } else {
29         r->l = merge(l, r->l); r->recalc(); return r;
30     }
31 }
32 Node* ins(Node* t, Node* n, int pos) {
33     auto [l,r] = split(t, pos);
34     return merge(merge(l, n), r);
35 }
36// Example application: move the range [l, r) to index k
37void move(Node*& t, int l, int r, int k) {
38     Node *a, *b, *c;
39     tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
40     if (k <= l) t = merge(ins(a, b, k), c);
41     else t = merge(a, ins(c, b, k - r));
42 }
```

FenwickTree.h

Description: Computes partial sums $a[0] + a[1] + \dots + a[\text{pos} - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value.

Time: Both operations are $\mathcal{O}(\log N)$.

e62fac, 22 lines

```
1 struct FT {
2     vector<ll> s;
```

```
3 FT(int n) : s(n) {}
4 void update(int pos, ll dif) { // a[pos] += dif
5     for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
6 }
7 ll query(int pos) { // sum of values in [0, pos)
8     ll res = 0;
9     for (; pos > 0; pos &= pos - 1) res += s[pos-1];
10    return res;
11 }
12 int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
13     // Returns n if no sum is >= sum, or -1 if empty sum is.
14     if (sum <= 0) return -1;
15     int pos = 0;
16     for (int pw = 1 << 25; pw; pw >= 1) {
17         if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
18             pos += pw, sum -= s[pos-1];
19     }
20     return pos;
21 }
22};
```

FenwickTree2d.h

Description: Computes sums $a[i,j]$ for all $i < l, j < j$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

FenwickTree.h 157f07, 22 lines

```
1 struct FT2 {
2     vector<vi> ys; vector<FT> ft;
3     FT2(int limx) : ys(limx) {}
4     void fakeUpdate(int x, int y) {
5         for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
6     }
7     void init() {
8         for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
9     }
10    int ind(int x, int y) {
11        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
12    void update(int x, int y, ll dif) {
13        for (; x < sz(ys); x |= x + 1)
14            ft[x].update(ind(x, y), dif);
15    }
16    ll query(int x, int y) {
17        ll sum = 0;
18        for (; x; x &= x - 1)
19            sum += ft[x-1].query(ind(x-1, y));
20        return sum;
21    }
22};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a + 1], \dots V[b - 1])$ in constant time.

Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);

Time: $\mathcal{O}(|V| \log |V| + Q)$

510c32, 16 lines

```
1 template<class T>
2 struct RMQ {
3     vector<vector<T>> jmp;
4     RMQ(const vector<T>& V) : jmp(1, V) {
5         for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
6             jmp.emplace_back(sz(V) - pw * 2 + 1);
7             rep(j,0,sz(jmp[k]))
8                 jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
9         }
10    }
11    T query(int a, int b) {
12        assert(a < b); // or return inf if a == b
13        int dep = 31 - __builtin_clz(b - a);
14        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
15    }
16};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).

Time: $\mathcal{O}(N\sqrt{Q})$

a12ef4, 49 lines

```
1 void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
2 void del(int ind, int end) { ... } // remove a[ind]
3 int calc() { ... } // compute current answer
4
5 vi mo(vector<pii> Q) {
6     int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
7     vi s(sz(Q)), res = s;
8     #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
9     iota(all(s), 0);
10    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
11    for (int qi : s) {
12        pii q = Q[qi];
13        while (L > q.first) add(--L, 0);
14        while (R < q.second) add(R++, 1);
15        while (L < q.first) del(L++, 0);
16        while (R > q.second) del(--R, 1);
17        res[qi] = calc();
18    }
19    return res;
20 }
21
22 vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
23     int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
24     vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
25     add(0, 0), in[0] = 1;
26     auto dfs = [&](int x, int p, int dep, auto& f) -> void {
27         par[x] = p;
28         L[x] = N;
29         if (dep) I[x] = N++;
30         for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
31         if (!dep) I[x] = N++;
32         R[x] = N;
33     };
34     dfs(root, -1, 0, dfs);
35     #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
36     iota(all(s), 0);
37     sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
38     for (int qi : s) rep(end,0,2) {
39         int &a = pos[end], b = Q[qi][end], i = 0;
40         #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
41             else { add(c, end); in[c] = 1; } a = c; }
42         while (!L[b] <= L[a] && R[a] <= R[b])
43             I[i++] = b, b = par[b];
44         while (a != b) step(par[a]);
45         while (i--) step(I[i]);
46         if (end) res[qi] = calc();
47     }
48     return res;
49 }
```

Numerical (3)

3.1 Polynomials and recurrences

Polynomial2.h

Description:

- inverse: Find a polynomial b so that $a(x)b(x) \equiv 1 \pmod{x^n \pmod{mod}}$. n must be a power of 2. The max length of the array should be at least twice the actual length.
- sqrt: Find a polynomial b so that $b^2(x) \equiv a(x) \pmod{x^n \pmod{mod}}$. $n \geq 2$ must be a power of 2. The max length of the array should be at least twice the actual length.
- divide: Given polynomial a and b with degree n and m respectively, find $a(x) = d(x)b(x) + r(x)$ with $\deg(d) \leq n - m$ and $\deg(r) < m$. The max length of the array should be at least four times the actual length.

a1204d, 77 lines
<pre>1template <int MAXN = 1000000> 2struct polynomial { 3 ntt<MAXN> tr; 4 5 void inverse(int *a, int *b, int n, int mod, int prt) { 6 static int c[MAXN]; 7 b[0] = ::inverse(a[0], mod); 8 b[1] = 0; 9 for (int m = 2, i; m <= n; m <= 1) { 10 copy(a, a + m, c); 11 fill(b + m, b + m + m, 0); 12 fill(c + m, c + m + m, 0); 13 tr.solve(c, m + m, 0, mod, prt); 14 tr.solve(b, m + m, 0, mod, prt); 15 for (int i = 0; i < m + m; ++i) 16 b[i] = 1LL * b[i] * 17 (2 - 1LL * b[i] * c[i] % mod + mod) % mod; 18 tr.solve(b, m + m, 1, mod, prt); 19 fill(b + m, b + m + m, 0); 20 } 21 } 22 23 void sqrt(int *a, int *b, int n, int mod, int prt) { 24 static int d[MAXN], ib[MAXN]; 25 b[0] = 1; 26 b[1] = 0; 27 int i2 = ::inverse(2, mod), m, i; 28 for (int m = 2; m <= n; m <= 1) { 29 copy(a, a + m, d); 30 fill(d + m, d + m + m, 0); 31 fill(b + m, b + m + m, 0); 32 tr.solve(d, m + m, 0, mod, prt); 33 inverse(b, ib, m, mod, prt); 34 tr.solve(ib, m + m, 0, mod, prt); 35 tr.solve(b, m + m, 0, mod, prt); 36 for (int i = 0; i < m + m; ++i) 37 b[i] = (1LL * b[i] * i2 + 38 1LL * i2 * d[i] % mod * ib[i]) % 39 mod; 40 tr.solve(b, m + m, 1, mod, prt); 41 fill(b + m, b + m + m, 0); 42 } 43 } 44 45 void divide(int *a, int n, int *b, int m, int *d, int *r, 46 int mod, int prt) { 47 static int u[MAXN], v[MAXN]; 48 while (!b[m - 1]) --m; 49 int p = 1, t = n - m + 1; 50 while (p < t < 1) p <= 1; 51 fill(u, u + p, 0); 52 reverse_copy(b, b + m, u); 53 inverse(u, v, p, mod, prt); 54 fill(v + t, v + p, 0); 55 tr.solve(v, p, 0, mod, prt); 56 reverse_copy(a, a + n, u); 57 fill(u + t, u + p, 0); 58 tr.solve(u, p, 0, mod, prt); 59 for (int i = 0; i < p; ++i) 60 u[i] = 1LL * u[i] * v[i] % mod; 61 tr.solve(u, p, 1, mod, prt); 62 reverse(u, u + t); 63 copy(u, u + t, d); 64 for (p = 1; p < n; p <= 1); 65 fill(u + t, u + p, 0); 66 tr.solve(u, p, 0, mod, prt); 67 copy(b, b + m, v); 68 fill(v + m, v + p, 0); 69 tr.solve(v, p, 0, mod, prt); 70 for (int i = 0; i < p; ++i) 71 u[i] = 1LL * u[i] * v[i] % mod; 72 tr.solve(u, p, 1, mod, prt); 73 for (int i = 0; i < m; ++i) 74 r[i] = (a[i] - u[i] + mod) % mod; 75 fill(r + m, r + p, 0); 76 } 77};</pre>

c9b7b0, 17 lines
<pre>1struct Poly { 2 vector<double> a; 3 double operator()(double x) const { 4 double val = 0; 5 for (int i = sz(a); i--;) (val += x) += a[i]; 6 return val; 7 } 8 void diff() { 9 rep(i,1,sz(a)) a[i-1] = i*a[i]; 10 a.pop_back(); 11 } 12 void divroot(double x0) { 13 double b = a.back(), c; a.back() = 0; 14 for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c; 15 a.pop_back(); 16 } 17};</pre>
PolyRoots.h Description: Finds the real roots to a polynomial. Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve $x^2-3x+2 = 0$ Time: $\mathcal{O}(n^2 \log(1/\epsilon))$
Polynomial.h
<pre>1vector<double> polyRoots(Poly p, double xmin, double xmax) { 2 if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; } 3 vector<double> ret; 4 Poly der = p; 5 der.diff(); 6 auto dr = polyRoots(der, xmin, xmax); 7 dr.push_back(xmin-1); dr.push_back(xmax+1); 8 sort(all(dr)); 9 rep(i,0,sz(dr)-1) { 10 double l = dr[i], h = dr[i+1]; 11 bool sign = p(l) > 0; 12 if (sign ^ (p(h) > 0)) { 13 rep(it,0,60) { // while (h - l > 1e-8) 14 double m = (l + h) / 2, f = p(m); 15 if ((f <= 0) ^ sign) l = m; 16 else h = m; 17 } 18 ret.push_back((l + h) / 2); 19 } } 20 return ret; 21}</pre>
b00bfe, 21 lines
PolyInterpolate.h Description: Given n points $(x[i], y[i])$, computes an $n-1$-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi)$, $k = 0 \dots n-1$. Time: $\mathcal{O}(n^2)$
08bf48, 13 lines
<pre>1typedef vector<double> vd; 2vd interpolate(vd x, vd y, int n) { 3 vd res(n), temp(n); 4 rep(k,0,n-1) rep(i,k+1,n) 5 y[i] = (y[i] - y[k]) / [x[i] - x[k]]; 6 double last = 0; temp[0] = 1; 7 rep(k,0,n) rep(i,0,n) { 8 res[i] += y[k] * temp[i]; 9 swap(last, temp[i]); 10 temp[i] -= last * x[k]; 11 } 12 return res; 13}</pre>
BerlekampMassey.h Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$. Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2} Time: $\mathcal{O}(N^2)$

ModPow.h
<pre>1const ll mod = 1000000007; 2vector<ll> berlekampMassey(vector<ll> s) { 3 int n = s.size(), L = 0, m = 0; 4 vector<ll> C(n), B(n), T; 5 C[0] = B[0] = 1; 6 7 ll b = 1; 8 rep(i, 0, n) { ++m; 9 ll d = s[i]%mod; 10 rep(j, 1, L) d = (d + C[j] * s[i - j])%mod; 11 if (!d) continue; 12 T = C; ll coef = d*modpow(b, mod-2)%mod; 13 rep(j, m, n) C[j] = (C[j] - coef * B[j - m])%mod; 14 if (2*L > i) continue; 15 L = i + 1 - L; B = T; b = d; m = 0; 16 } 17 18 C.resize(L+1); C.erase(C.begin()); 19 for (ll& x : C) x = (mod - x)%mod; 20 return C; 21}</pre>
12a403, 21 lines
LinearRecurrence.h Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey. Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number Time: $\mathcal{O}(n^2 \log k)$
f4e444, 26 lines
<pre>1typedef vector<ll> Poly; 2ll linearRec(Poly S, Poly tr, ll k) { 3 int n = sz(tr); 4 5 auto combine = [&](Poly a, Poly b) { 6 Poly res(n * 2 + 1); 7 rep(i,0,n+1) rep(j,0,n+1) 8 res[i + j] = (res[i + j] + a[i] * b[j]) % mod; 9 for (int i = 2 * n; i > n; --i) rep(j,0,n) 10 res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod; 11 res.resize(n + 1); 12 return res; 13 }; 14 15 Poly pol(n + 1), e(pol); 16 pol[0] = e[1] = 1; 17 18 for (++k; k; k /= 2) { 19 if (k % 2) pol = combine(pol, e); 20 e = combine(e, e); 21 } 22 23 ll res = 0; 24 rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod; 25 return res; 26}</pre>
3.2 Linear Algebra
IntDet.h Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version. Time: $\mathcal{O}(N^3)$
ab0407, 18 lines
<pre>1const ll mod = 1000000007; 2 3ll det(vector<vector<ll>>& a) { 4 int n = sz(a); ll ans = 1; 5 rep(i, 0, n) { 6 rep(j, i+1, n) 7 while (a[j][i] != 0) { // gcd step 8 ll t = a[i][i] / a[j][i];</pre>

```
9         if(t) rep(k, i, n)
10             a[i][k] = (a[i][k] - a[j][k] * t) % mod;
11         swap(a[i], a[j]);
12         ans *= -1;
13     }
14     ans = ans * a[i][i] % mod;
15     if (!ans) return 0;
16 }
17 return (ans + mod) % mod;
18}
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
1typedef vector<double> vd;
2const double eps = 1e-12;
3
4int solveLinear(vector<vd>& A, vd& b, vd& x) {
5    int n = sz(A), m = sz(x), rank = 0, br, bc;
6    if (n) assert(sz(A[0]) == m);
7    vi col(m); iota(all(col), 0);
8
9    rep(i,0,n) {
10        double v, bv = 0;
11        rep(r,i,n) rep(c,i,m)
12            if ((v = fabs(A[r][c])) > bv)
13                br = r, bc = c, bv = v;
14        if (bv <= eps) {
15            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
16            break;
17        }
18        swap(A[i], A[br]);
19        swap(b[i], b[br]);
20        swap(col[i], col[bc]);
21        rep(j,0,n) swap(A[j][i], A[j][bc]);
22        bv = 1/A[i][i];
23        rep(j,i+1,n) {
24            double fac = A[j][i] * bv;
25            b[j] -= fac * b[i];
26            rep(k,i+1,m) A[j][k] -= fac*A[i][k];
27        }
28        rank++;
29    }
30
31    x.assign(m, 0);
32    for (int i = rank; i--;) {
33        b[i] /= A[i][i];
34        x[col[i]] = b[i];
35        rep(j,0,i) b[j] -= A[j][i] * b[i];
36    }
37    return rank; // (multiple solutions if rank < m)
38}
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```
SolveLinear.h
08e495, 7 lines

1rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
2// ... then at the end:
3x.assign(m, undefined);
4rep(i,0,rank) {
5    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
6    x[col[i]] = b[i] / A[i][i];
7fail:; }
```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
1typedef bitset<1000> bs;
2
3int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
```

```
4    int n = sz(A), rank = 0, br;
5    assert(m <= sz(x));
6    vi col(m); iota(all(col), 0);
7    rep(i,0,n) {
8        for (br=i; br<n; ++br) if (A[br].any()) break;
9        if (br == n) {
10            rep(j,i,n) if(b[j]) return -1;
11            break;
12        }
13        int bc = (int)A[br]._Find_next(i-1);
14        swap(A[i], A[br]);
15        swap(b[i], b[br]);
16        swap(col[i], col[bc]);
17        rep(j,0,n) if (A[j][i] != A[j][bc]) {
18            A[j].flip(i); A[j].flip(bc);
19        }
20        rep(j,i+1,n) if (A[j][i]) {
21            b[j] ^= b[i];
22            A[j] ^= A[i];
23        }
24        rank++;
25    }
26
27    x = bs();
28    for (int i = rank; i--;) {
29        if (!b[i]) continue;
30        x[col[i]] = 1;
31        rep(j,0,i) b[j] ^= A[j][i];
32    }
33    return rank; // (multiple solutions if rank < m)
34}
```

MatrixInverse.h
Description: Invert matrix A . Returns rank; result is stored in A unless singular ($\text{rank} < n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

```
1int matInv(vector<vector<double>>& A) {
2    int n = sz(A); vi col(n);
3    vector<vector<double>> tmp(n, vector<double>(n));
4    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
5
6    rep(i,0,n) {
7        int r = i, c = i;
8        rep(j,i,n) rep(k,i,n)
9            if (fabs(A[j][k]) > fabs(A[r][c]))
10                r = j, c = k;
11        if (fabs(A[r][c]) < 1e-12) return i;
12        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
13        rep(j,0,n)
14            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
15        swap(col[i], col[c]);
16        double v = A[i][i];
17        rep(j,i+1,n) {
18            double f = A[j][i] / v;
19            A[j][i] = 0;
20            rep(k,i+1,n) A[j][k] -= f*A[i][k];
21            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
22        }
23        rep(j,i+1,n) A[i][j] /= v;
24        rep(j,0,n) tmp[i][j] /= v;
25        A[i][i] = 1;
26    }
27
28    for (int i = n-1; i > 0; --i) rep(j,0,i) {
29        double v = A[j][i];
30        rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
31    }
32
33    rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
34    return n;
35}
```

3.3 Optimization
GoldenSectionSearch.h
Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.
Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

```
1double gss(double a, double b, double (*f)(double)) {
2    double r = (sqrt(5)-1)/2, eps = 1e-7;
3    double x1 = b - r*(b-a), x2 = a + r*(b-a);
4    double f1 = f(x1), f2 = f(x2);
5    while (b-a > eps)
6        if (f1 < f2) { //change to > to find maximum
7            b = x2; x2 = x1; f2 = f1;
8            x1 = b - r*(b-a); f1 = f(x1);
9        } else {
10            a = x1; x1 = x2; f1 = f2;
11            x2 = a + r*(b-a); f2 = f(x2);
12        }
13    return a;
14}
```

IntegrateAdaptive.h
Description: Fast integration using an adaptive Simpson's rule.
Usage: double sphereVolume = quad(-1, 1, [](double x) { return quad(-1, 1, [&](double y) { return quad(-1, 1, [&](double z) { return x*x + y*y + z*z < 1; }}});});

```
1typedef double d;
2#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
3
4template <class F>
5d rec(F& f, d a, d b, d eps, d S) {
6    d c = (a + b) / 2;
7    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
8    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
9        return T + (T - S) / 15;
10    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
11}
12template<class F>
13d quad(d a, d b, F f, d eps = 1e-8) {
14    return rec(f, a, b, eps, S(a, b));
15}
```

Simplex.h
Description: Solves a general linear maximization problem: maximize c^Tx subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of c^Tx otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM \times \text{\#piv})$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in general.

```
1typedef double T; // long double, Rational, double + mod<P>...
2typedef vector<T> vd;
3typedef vector<vd> vvd;
4
5const T eps = 1e-8, inf = 1/.0;
6#define MP make_pair
7#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
8
9struct LPSolver {
10    int m, n; vi N, B; vvd D;
```

```
11
12 LPSolver(const vvd& A, const vvd& b, const vvd& c) :
13     m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
14     rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
15     rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
16     rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
17     N[n] = -1; D[m+1][n] = 1;
18 }
19
20 void pivot(int r, int s) {
21     T *a = D[r].data(), inv = 1 / a[s];
22     rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
23         T *b = D[i].data(), inv2 = b[s] * inv;
24         rep(j,0,n+2) b[j] -= a[j] * inv2;
25         b[s] = a[s] * inv2;
26     }
27     rep(j,0,n+2) if (j != s) D[r][j] *= inv;
28     rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
29     D[r][s] = inv;
30     swap(B[r], N[s]);
31 }
32
33 bool simplex(int phase) {
34     int x = m + phase - 1;
35     for (;;) {
36         int s = -1;
37         rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
38         if (D[x][s] >= -eps) return true;
39         int r = -1;
40         rep(i,0,m) {
41             if (D[i][s] <= eps) continue;
42             if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
43                 < MP(D[r][n+1] / D[r][s], B[r])) r = i;
44         }
45         if (r == -1) return false;
46         pivot(r, s);
47     }
48 }
49
50 T solve(vd &x) {
51     int r = 0;
52     rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
53     if (D[r][n+1] < -eps) {
54         pivot(r, n);
55         if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
56         rep(i,0,m) if (B[i] == -1) {
57             int s = 0;
58             rep(j,1,n+1) ltj(D[i]);
59             pivot(i, s);
60         }
61     }
62     bool ok = simplex(1); x = vd(n);
63     rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
64     return ok ? D[m][n+1] : inf;
65 }
66};
```

3.4 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_j^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\tilde{O}$ s for $N = 2^{22}$)

00ced6, 35 lines

```
1 typedef complex<double> C;
2 typedef vector<double> vd;
3 void fft(vector<C>& a) {
4     int n = sz(a), L = 31 - __builtin_clz(n);
5     static vector<complex<long double>> R(2, 1);
6     static vector<C> rt(2, 1); // (^ 10% faster if double)
7     for (static int k = 2; k < n; k *= 2) {
8         R.resize(n); rt.resize(n);
9         auto x = polar(1.0/L, acos(-1.0/L) / k);
```

```
10     rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
11 }
12 vi rev(n);
13 rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
14 rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
15 for (int k = 1; k < n; k *= 2)
16     for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
17         C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
18         a[i + j + k] = a[i + j] - z;
19         a[i + j] += z;
20     }
21 }
22 vd conv(const vvd& a, const vvd& b) {
23     if (a.empty() || b.empty()) return {};
24     vd res(sz(a) + sz(b) - 1);
25     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
26     vector<C> in(n), out(n);
27     copy(all(a), begin(in));
28     rep(i,0,sz(b)) in[i].imag(b[i]);
29     fft(in);
30     for (C& x : in) x *= x;
31     rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
32     fft(out);
33     rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
34     return res;
35 }
```

NumberTheoreticTransform.h

Description: NTT convolution modulo 998244353

Usage: Remember to call `init()` at the start, make sure polynomials are nonempty

ModPow.h

e63a4a, 39 lines

```
1 typedef vector<ll> vll;
2 const ll mod = 998244353;
3 ll W[23];
4 void init(){
5     W[0] = 2200; //order 2^23
6     for(int i = 0; i < 22; i++) W[i+1] = W[i]*W[i]%mod;
7 }
8 vll dft(vll poly, int b){
9     int n = 1 << b;
10    poly.resize(n, 0);
11    for(int loop = 1; loop <= b; loop++){
12        vll nv(n, 0);
13        ll acc = 1;
14        int space = n >> loop;
15        for(int q = 0; q < n; q += space){
16            for(int r = 0; r < space; r++){
17                nv[q+r] = (poly[2*q*n + r]
18                    + acc*poly[2*q*n + space + r])%mod;
19                acc = acc*W[23-loop]%mod;
20            }
21            poly = nv;
22        }
23        return poly;
24    }
25 vll mul(vll a, vll b){
26     int sz = a.size()+b.size()-1;
27     int j = 1;
28     while(1 << j < sz) j++;
29
30     vll A = dft(a, j);
31     vll B = dft(b, j);
32     ll inv = modpow(1 << j, mod-2);
33     for(int i = 1 << j; i--;) B[i] = A[i]*B[i]%mod*inv%mod;
34
35     A = dft(B, j);
36     reverse(A.begin()+1, A.end());
37     A.resize(sz);
38     return A;
39 }
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
1 void FST(vi& a, bool inv) {
2     for (int n = sz(a), step = 1; step < n; step *= 2) {
3         for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
4             int &u = a[j], &v = a[j + step]; tie(u, v) =
5                 inv ? pii(v - u, u) : pii(v, u + v); // AND
6                 inv ? pii(v, u - v) : pii(u + v, u); // OR
7                 pii(u + v, u - v); // XOR
8         }
9     }
10    if (inv) for (int& x : a) x /= sz(a); // XOR only
11 }
12 vi conv(vi a, vi b) {
13     FST(a, 0); FST(b, 0);
14     rep(i,0,sz(a)) a[i] *= b[i];
15     FST(a, 1); return a;
16 }
```

Number theory (4)

4.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set `mod` to some number first and then you can use the structure.

euclid.h

35bfea, 18 lines

```
1 const ll mod = 17; // change to something else
2 struct Mod {
3     ll x;
4     Mod(ll xx) : x(xx) {}
5     Mod operator+(Mod b) { return Mod((x + b.x) % mod); }
6     Mod operator-(Mod b) { return Mod((x - b.x + mod) % mod); }
7     Mod operator*(Mod b) { return Mod((x * b.x) % mod); }
8     Mod operator/(Mod b) { return *this * invert(b); }
9     Mod invert(Mod a) {
10         ll x, y, g = euclid(a.x, mod, x, y);
11         assert(g == 1); return Mod((x + mod) % mod);
12     }
13     Mod operator^(ll e) {
14         if (!e) return Mod(1);
15         Mod r = *this ^ (e / 2); r = r * r;
16         return e&1 ? *this * r : r;
17     }
18 };
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes $\text{LIM} \leq \text{mod}$ and that `mod` is a prime.

6f684f, 3 lines

```
1 const ll mod = 1000000007, LIM = 200000;
2 ll* inv = new ll[LIM] - 1; inv[1] = 1;
3 rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

b83e45, 8 lines

```
1 const ll mod = 1000000007; // faster if const
2
3 ll modpow(ll b, ll e) {
4     ll ans = 1;
5     for (; e; b = b * b % mod, e /= 2)
6         if (e & 1) ans = ans * b % mod;
7     return ans;
8 }
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. `modLog(a,1,m)` can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

c40b8, 11 lines

```
1 ll modLog(ll a, ll b, ll m) {
2     ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
```



```
3 unordered_map<ll, ll> A;
4 while (j <= n && (e = f * e * a % m) != b % m)
5     A[e * b % m] = j++;
6 if (e == b % m) return j;
7 if (__gcd(m, e) == __gcd(m, b))
8     rep(i,2,n+2) if (A.count(e = e * f % m))
9         return n * i - A[e];
10 return -1;
11}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

modsum(to, c, k, m) = $\sum_{i=0}^{t_0-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

5c5bc5, 16 lines

```
1 typedef unsigned long long ull;
2 ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
3
4 ull divsum(ull to, ull c, ull k, ull m) {
5     ull res = k / m * sumsq(to) + c / m * to;
6     k %= m; c %= m;
7     if (!k) return res;
8     ull to2 = (to * k + c) / m;
9     return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
10 }
11
12 ll modsum(ull to, ll c, ll k, ll m) {
13     c = ((c % m) + m) % m;
14     k = ((k % m) + m) % m;
15     return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
16 }
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines

```
1 typedef unsigned long long ull;
2 ull modmul(ull a, ull b, ull M) {
3     ll ret = a * b - M * ull(1.L / M * a * b);
4     return ret + M * (ret < 0) - M * (ret >= (ll)M);
5 }
6 ull modpow(ull b, ull e, ull mod) {
7     ull ans = 1;
8     for (; e; b = modmul(b, b, mod), e /= 2)
9         if (e & 1) ans = modmul(ans, b, mod);
10 return ans;
11 }
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

ModPow.h 19a793, 24 lines

```
1 ll sqrt(ll a, ll p) {
2     a %= p; if (a < 0) a += p;
3     if (a == 0) return 0;
4     assert(modpow(a, (p-1)/2, p) == 1); // else no solution
5     if (p % 4 == 3) return modpow(a, (p+1)/4, p);
6     // a^(n+3)/8 or 2^(n+3)/8 * 2^a(n-1)/4 works if p % 8 == 5
7     ll s = p - 1, n = 2;
8     int r = 0, m;
9     while (s % 2 == 0)
10         ++r, s /= 2;
11 while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
12 ll x = modpow(a, (s + 1) / 2, p);
13 ll b = modpow(a, s, p), g = modpow(n, s, p);
14 for (; r = m) {
15     ll t = b;
16     for (m = 0; m < r && t != 1; ++m)
17         t = t * t % p;
18     if (m == 0) return x;
19     ll gs = modpow(g, 1LL << (r - m - 1), p);
20     g = gs * gs % p;
```

```
21     x = x * gs % p;
22     b = b * g % p;
23 }
24 }
```

4.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx$ 1.5s

6b2912, 20 lines

```
1 const int LIM = 1e6;
2 bitset<LIM> isPrime;
3 vi eratosthenes() {
4     const int S = (int)round(sqrt(LIM)), R = LIM / 2;
5     vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
6     vector<pii> cp;
7     for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
8         cp.push_back({i, i * i / 2});
9         for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
10    }
11    for (int L = 1; L <= R; L += S) {
12        array<bool, S> block{};
13        for (auto &[p, idx] : cp)
14            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
15        rep(i,0,min(S, R - L))
16            if (!block[i]) pr.push_back((L + i) * 2 + 1);
17    }
18    for (int i : pr) isPrime[i] = 1;
19    return pr;
20 }
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

ModMuLL.h 60dcd1, 12 lines

```
1 bool isPrime(ull n) {
2     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
3     ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
4         s = __builtin_ctzll(n-1), d = n >> s;
5     for (ull a : A) { // a count trailing zeroes
6         ull p = modpow(a%n, d, n), i = s;
7         while (p != 1 && p != n - 1 && a % n && i--)
8             p = modmul(p, p, n);
9         if (p != n-1 && i != s) return 0;
10    }
11    return 1;
12 }
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

ModMuLL.h, MillerRabin.h d8d98d, 18 lines

```
1 ull pollard(ull n) {
2     ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
3     auto f = [&](ull x) { return modmul(x, x, n) + i; };
4     while (t++ % 40 || __gcd(prd, n) == 1) {
5         if (x == y) x = ++i, y = f(x);
6         if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
7         x = f(x), y = f(f(y));
8     }
9     return __gcd(prd, n);
10 }
11 vector<ull> factor(ull n) {
12     if (n == 1) return {};
13     if (isPrime(n)) return {n};
14     ull x = pollard(n);
15     auto l = factor(x), r = factor(n / x);
16     l.insert(l.end(), all(r));
17     return l;
18 }
```

4.3 Divisibility

Euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
1 ll euclid(ll a, ll b, ll &x, ll &y) {
2     if (!b) return x = 1, y = 0, a;
3     ll d = euclid(b, a % b, y, x);
4     return y -= a/b * x, d;
5 }
```

CRT.h

Description: Chinese Remainder Theorem.

crt(a, m, b, n) computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

euclid.h 04d93a, 7 lines

```
1 ll crt(ll a, ll m, ll b, ll n) {
2     if (n > m) swap(a, b), swap(m, n);
3     ll x, y, g = euclid(m, n, x, y);
4     assert((a - b) % g == 0); // else no solution
5     x = (b - a) % n * x % n / g * m + a;
6     return x < 0 ? x + m*n/g : x;
7 }
```

4.3.1 Bézout's identity

For $a \neq b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to $ax + by = d$. If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

PhiFunction.h

Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p \text{ prime} \Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, $m, n \text{ coprime} \Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$, then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.

$$\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$$

Euler's thm: $a, n \text{ coprime} \Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.

Fermat's little thm: $p \text{ prime} \Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

cf7d6d, 8 lines

```
1 const int LIM = 5000000;
2 int phi[LIM];
3
4 void calculatePhi() {
5     rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
6     for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
7         for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
8 }
```

EuclideanLikeAlgorithm.h

Description: Compute $\sum_{i=0}^{n-1} \lfloor \frac{a+bi}{m} \rfloor$.

36b884, 8 lines

```
1 ll solve(ll n, ll a, ll b, ll m) {
2     if (b == 0) return n * (a / m);
3     if (a >= m) return n * (a / m) + solve(n, a % m, b, m);
4     if (b >= m)
5         return (n - 1) * n / 2 * (b / m) +
6             solve(n, a, b % m, m);
7     return solve((a + b * n) / m, (a + b * n) % m, m, b);
8 }
```


4.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a 's eventually become cyclic.

Time: $\mathcal{O}(\log N)$

```
dd6c5e, 20 lines
1typedef double d; // for N ~ 1e7; long double for N ~ 1e9
2pair<ll, ll> approximate(d x, ll N) {
3    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
4    for (;;) {
5        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
6            a = (ll)floor(y), b = min(a, lim),
7            NP = b*P + LP, NQ = b*Q + LQ;
8        if (a > b) {
9            // If b > a/2, we have a semi-convergent that gives us a
10           // better approximation; if b = a/2, we *may* have one.
11           // Return {P, Q} here for a more canonical approximation.
12           return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
13               make_pair(NP, NQ) : make_pair(P, Q);
14        }
15        if (abs(y = 1/(y - (d)a)) > 3*N) {
16            return {NP, NQ};
17        }
18        LP = P; P = NP; LQ = Q; Q = NQ;
19    }
20}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}

Time: $\mathcal{O}(\log(N))$

```
27ab3e, 23 lines
1struct Frac { ll p, q; };
2
3template<class F>
4Frac fracBS(F f, ll N) {
5    bool dir = 1, A = 1, B = 1;
6    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
7    if (f(lo)) return lo;
8    assert(f(hi));
9    while (A || B) {
10        ll adv = 0, step = 1; // move hi if dir, else lo
11        for (int si = 0; step; (step *= 2) >= si) {
12            adv += step;
13            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
14            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
15                adv -= step; si = 2;
16            }
17        }
18        hi.p += lo.p * adv; hi.q += lo.q * adv;
19        dir = !dir; swap(lo, hi);
20        A = B; B = !adv;
21    }
22    return dir ? hi : lo;
23}
```

4.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

4.6 Primes

$p = 962592769$ is such that $2^{2^{21}} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.7 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

4.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

DirichletConvolution.h

Description: Define the Dirichlet convolution $f * g(n)$ as:

$$f * g(n) = \sum_{d=1}^n [d|n] f(n)g(\frac{n}{d})$$

Assume we are going to calculate some function $S(n) = \sum_{i=1}^n f(i)$, where $f(n)$ is a multiplicative function. Say we find some $g(n)$ that is simple to calculate, and $\sum_{i=1}^n f * g(i)$ can be figured out in $O(1)$ complexity. Then we have

$$\begin{aligned} \sum_{i=1}^n f * g(i) &= \sum_{i=1}^n \sum_d [d|i] g(\frac{i}{d}) f(d) = \sum_{\frac{i}{d}=1}^n \sum_{d=1}^{\lfloor \frac{n}{i} \rfloor} g(\frac{i}{d}) f(d) \\ &= \sum_{i=1}^n \sum_{d=1}^{\lfloor \frac{n}{i} \rfloor} g(i) f(d) = g(1)S(n) + \sum_{i=2}^n g(i)S(\lfloor \frac{n}{i} \rfloor), \\ S(n) &= \frac{\sum_{i=1}^n f * g(i) - \sum_{i=2}^n g(i)S(\lfloor \frac{n}{i} \rfloor)}{g(1)}. \end{aligned}$$

It can be proven that $\lfloor \frac{n}{i} \rfloor$ has at most $O(\sqrt{n})$ possible values. Therefore, the calculation of $S(n)$ can be reduced to $O(\sqrt{n})$ calculations of $S(\lfloor \frac{n}{i} \rfloor)$. By applying the master theorem, it can be shown that the complexity of such method is $O(n^{\frac{3}{4}})$.

Moreover, since $f(n)$ is multiplicative, we can process the first $n^{\frac{2}{3}}$ elements via linear sieve, and for the rest of the elements, we apply the method shown above. The complexity can thus be enhanced to $O(n^{\frac{2}{3}})$.

For the prefix sum of Euler's function $S(n) = \sum_{i=1}^n \varphi(i)$, notice that $\sum_{d|n} \varphi(d) = n$. Hence $\varphi * I = id$. ($I(n) = 1, id(n) = n$) Now let $g(n) = I(n)$, and we have $S(n) = \sum_{i=1}^n i - \sum_{i=2}^n S(\lfloor \frac{n}{i} \rfloor)$.

For the prefix sum of Mobius function $S(n) = \sum_{i=1}^n \mu(i)$, notice that $\mu * I = (n)\{[n = 1]\}$. Hence $S(n) = 1 - \sum_{i=2}^n S(\lfloor \frac{n}{i} \rfloor)$.

Some other convolutions include $(p^k)\{1 - p\} * id = I, (p^k)\{p^k - p^{k+1}\} * id^2 = id$ and $(p^k)\{p^{2k} - p^{2k-2}\} * I = id^2$.

1. CUBEN should be $N^{\frac{1}{3}}$.
2. Pass p_f that returns the prefix sum of $f(x) (1 \leq x < th)$.
3. Pass p_g that returns the prefix sum of $g(x) (0 \leq x \leq N)$.
4. Pass p_c that returns the prefix sum of $f * g(x) (0 \leq x \leq N)$.

5. Pass th as the threshold, which generally should be $N^{\frac{2}{3}}$.
6. Pass mod as the module number, inv as the inverse of $g(1)$ regarding mod.
7. **Remember that x in p_g(x) and p_c(x) may be larger than mod!**
8. Run init(n) first.
9. Use ans(x) to fetch answer for $\frac{n}{x}$.

```
d25935, 29 lines
1template <int CUBEN = 3000>
2struct prefix_mul {
3    typedef ll (*func)(ll);
4    func p_f, p_g, p_c;
5    ll mod, th, inv, n, mem[CUBEN];
6
7    prefix_mul(func p_f, func p_g, func p_c, ll th, ll mod, ll
8        ↪ inv)
9        : p_f(p_f), p_g(p_g), p_c(p_c), th(th), mod(mod), inv(inv)
10        ↪ {}
11
12    void init(ll n) {
13        prefix_mul::n = n;
14        for (ll i = 1, la; i <= n; i = la + 1) {
15            if ((la = n / (n / i)) < th) continue;
16            ll &ans = mem[n / la] = p_c(la);
17            for (ll j = 2, ne; j <= la; j = ne + 1) {
18                ne = la / (la / j);
19                ans = (ans + mod - (p_g(ne) - p_g(j - 1) + mod)
20                    * (la / j < th ? p_f(la / j) : mem[n / (la / j)])) %
21                    ↪ mod;
22                if (ans >= mod) ans -= mod;
23            }
24            if (inv != 1) ans = ans * inv % mod;
25        }
26        ll ans(ll x) {
27            if (n / x < th) return p_f(n / x);
28            return mem[n / (n / x)];
29        }
30};
```

Graph (5)

5.1 Fundamentals

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j] = \text{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or $-\text{inf}$ if the path goes through a negative-weight cycle.

Time: $\mathcal{O}(N^3)$

```
531245, 12 lines
1const ll inf = 1LL << 62;
2void floydWarshall(vector<vector<ll>>& m) {
3    int n = sz(m);
4    rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
5    rep(k,0,n) rep(i,0,n) rep(j,0,n)
6        if (m[i][k] != inf && m[k][j] != inf) {
7            auto newDist = max(m[i][k] + m[k][j], -inf);
8            m[i][j] = min(m[i][j], newDist);
9        }
10    rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
11        if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
12}
```

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max |w_i| < \sim 2^{63}$.

Time: $\mathcal{O}(VE)$

```
830a8f, 23 lines
1const ll inf = LLONG_MAX;
2struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
3struct Node { ll dist = inf; int prev = -1; };
4
5void bellmanFord(vector<Node>&& nodes, vector<Ed>&& eds, int s) {
6    nodes[s].dist = 0;
7    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
8
9    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
10    rep(i,0,lim) for (Ed ed : eds) {
```

```
11 Node cur = nodes[ed.a], &dest = nodes[ed.b];
12 if (abs(cur.dist) == inf) continue;
13 ll d = cur.dist + ed.w;
14 if (d < dest.dist) {
15     dest.prev = ed.a;
16     dest.dist = (i < lim-1 ? d : -inf);
17 }
18 }
19 rep(i,0,lim) for (Ed e : eds) {
20     if (nodes[e.a].dist == -inf)
21         nodes[e.b].dist = -inf;
22 }
23 }
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.

Time: $\mathcal{O}(|V| + |E|)$

d678d8, 8 lines

```
1vi topoSort(const vector<vi>& gr) {
2    vi indeg(sz(gr)), q;
3    for (auto& li : gr) for (int x : li) indeg[x]++;
4    rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
5    rep(j,0,sz(q)) for (int x : gr[q[j]])
6        if (--indeg[x] == 0) q.push_back(x);
7    return q;
8}
```

5.2 Network flow

Dinic.h

Description: $\mathcal{O}(VE \log U)$ where $U = \max |\text{cap}|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching.

93cc2c, 48 lines

```
1const int N = 1000, M = 100000;
2struct E {
3    int sz, begin[N], dest[M], next[M]; ll flow[M];
4    void clear(int n) { sz = 0; fill(begin, begin+n, -1); }
5    E(int n = N) { clear(n); }
6    void add_edge(int u, int v, ll f) {
7        dest[sz] = v; next[sz] = begin[u];
8        flow[sz] = f; begin[u] = sz++;
9        dest[sz] = u; next[sz] = begin[v];
10       flow[sz] = 0; begin[v] = sz++;
11   }
12};
13struct Dinic {
14    int n, s, t, d[N], w[N], q[N];
15    int bfs(E& e) {
16        fill(d, d+n, -1); int l, r;
17        q[l = r = 0] = s, d[s] = 0;
18        for(; l <= r; ++l)
19            for(int k = e.begin[q[l]]; ~k; k = e.next[k])
20                if(!d[e.dest[k]] && e.flow[k] > 0)
21                    d[e.dest[k]] = d[q[l]] + 1, q[++r] = e.dest[k];
22        return ~d[t] ? 1 : 0;
23   }
24   int dfs(E& e, int u, ll ext) {
25       if(u == t) return ext;
26       int k = w[u]; ll ret = 0;
27       for(; ~k; k = e.next[k], w[u] = k) {
28           if(!ext) break;
29           if(d[e.dest[k]] == d[u]+1 && e.flow[k] > 0) {
30               ll flow = dfs(e, e.dest[k], min(e.flow[k], ext));
31               if(flow > 0) {
32                   e.flow[k] -= flow, e.flow[k^1] += flow;
33                   ret += flow, ext -= flow;
34               }
35           }
36       }
37       if(!~k) d[u] = -1;
38       return ret;
39   }
40   int solve(E& e, int n_, int s_, int t_) {
41       ll ans = 0; n = n_, s = s_, t = t_;
```

```
42 while(bfs(e)) {
43     rep(i, 0, n) w[i] = e.begin[i];
44     ans += dfs(e, s, LLONG_MAX);
45 }
46 return ans;
47 }
48};
```

MCMFEK.h

Description: Min-cost max-flow. EK is better for sparse graphs

c0f68f, 46 lines

```
1template <int MAXN = 1000, int MAXM = 100000>
2struct MCMF_EK {
3    struct EdgeList {
4        int size, begin[MAXN], dest[MAXM], next[MAXM], cost[MAXM], flow[
5            ↪ MAXM];
6        void clear(int n) { size = 0; fill(begin, begin + n, -1); }
7        EdgeList(int n = MAXN) { clear(n); }
8        void addEdge(int u, int v, int c, int f) {
9            dest[size] = v; next[size] = begin[u];
10           cost[size] = c; flow[size] = f; begin[u] = size++;
11           dest[size] = u; next[size] = begin[v];
12           cost[size] = -c; flow[size] = 0; begin[v] = size++;
13       }
14       int n, s, t, prev[MAXN], dist[MAXN], occur[MAXN];
15       bool augment(EdgeList &e) {
16           vector<int> queue; queue.push_back(s);
17           fill(dist, dist + n, INF); dist[s] = 0;
18           fill(occur, occur + n, 0); occur[s] = true;
19           for (int head = 0; head < (int)queue.size(); ++head) {
20               int x = queue[head];
21               for (int i = e.begin[x]; ~i; i = e.next[i]) {
22                   int y = e.dest[i];
23                   if (e.flow[i] && dist[y] > dist[x] + e.cost[i]) {
24                       dist[y] = dist[x] + e.cost[i]; prev[y] = i;
25                       if (!occur[y]) {
26                           occur[y] = true; queue.push_back(y);
27                       }
28                   }
29               }
30               occur[x] = false;
31           }
32           pair<int, int> solve( EdgeList &e, int n_, int s_, int t_) {
33               n = n_; s = s_; t = t_;
34               pair<int, int> ans = make_pair(0, 0);
35               while (augment(e)) {
36                   int num = INF;
37                   for (int i = t; i != s; i = e.dest[prev[i] ^ 1])
38                       num = min(num, e.flow[prev[i]]);
39                   ans.first += num;
40                   for (int i = t; i != s; i = e.dest[prev[i] ^ 1]) {
41                       e.flow[prev[i]] -= num; e.flow[prev[i] ^ 1] += num;
42                       ans.second += num * e.cost[prev[i]];
43                   }
44                   return ans;
45               }
46           }
```

MCMFZKW.h

Description: Min-cost max-flow. ZKW is better for dense graphs.

64cee5, 43 lines

```
1template <int MAXN = 1000, int MAXM = 100000>
2struct MCMF_ZKW {
3    // struct EdgeList { ... };
4    // see MCMF_EK
5    int n, s, t, tf, tc, dis[MAXN], slack[MAXN], visit[MAXN];
6    int modTable() {
7        int delta = INF;
8        rep(i, 0, n) {
9            if (!visit[i] && slack[i] < delta) delta = slack[i];
10           slack[i] = INF;
11       }
12       if (delta == INF) return 1;
13       rep(i, 0, n) if (visit[i]) dis[i] += delta;
14       return 0;
15   }
```

```
16 int dfs(EdgeList &e, int x, int flow) {
17     if (x == t) {
18         tf += flow; tc += flow * (dis[s] - dis[t]);
19         return flow;
20     }
21     visit[x] = 1;
22     int left = flow;
23     for (int i = e.begin[x]; ~i; i = e.next[i])
24         if (e.flow[i] > 0 && !visit[e.dest[i]]) {
25             int y = e.dest[i];
26             if (dis[y] + e.cost[i] == dis[x]) {
27                 int delta = dfs(e, y, min(left, e.flow[i]));
28                 e.flow[i] -= delta; e.flow[i ^ 1] += delta; left -=
29                     ↪ delta;
30                 if (!left) { visit[x] = false; return flow; }
31             } else {
32                 slack[y] = min(slack[y], dis[y] + e.cost[i] - dis[x]);
33             }
34         }
35     return flow - left;
36 }
37 pair<int, int> solve( EdgeList &e, int n_, int s_, int t_) {
38     n = n_; s = s_; t = t_;
39     tf = tc = 0; fill(dis + 1, dis + n + 1, 0);
40     do { do {
41         fill(visit + 1, visit + n + 1, 0);
42         } while (dfs(e, s, INF)); } while (!modTable());
43     return make_pair(tf, tc);
44 }
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

162156, 25 lines

```
1template<class T> T edmondsKarp(vector<unordered_map<int, T>&&
2    graph, int source, int sink) {
3    T flow = 0; vi par(sz(graph)), q = par;
4    for (;) {
5        fill(all(par), -1); par[source] = 0;
6        int ptr = 1; q[0] = source;
7        rep(i,0,ptr) {
8            int x = q[i];
9            for (auto e : graph[x]) {
10               if (par[e.first] == -1 && e.second > 0) {
11                   par[e.first] = x; q[ptr++] = e.first;
12                   if (e.first == sink) goto out;
13               }
14           }
15           return flow;
16       }
17       T inc = numeric_limits<T>::max();
18       for (int y = sink; y != source; y = par[y])
19           inc = min(inc, graph[par[y]][y]);
20       flow += inc;
21       for (int y = sink; y != source; y = par[y]) {
22           int p = par[y];
23           if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
24           graph[y][p] += inc;
25       }
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Time: $\mathcal{O}(V^3)$

8b0e19, 19 lines

```
1pair<int, vi> globalMinCut(vector<vi> mat) {
2    pair<int, vi> best = {INT_MAX, {}}; int n = sz(mat);
3    vector<vi> co(n); rep(i,0,n) co[i] = {i};
4    rep(ph,1,n) {
```

```
5   vi w = mat[0];
6   size_t s = 0, t = 0;
7   rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
8       w[t] = INT_MIN;
9       s = t, t = max_element(all(w)) - w.begin();
10      rep(i,0,n) w[i] += mat[t][i];
11  }
12  best = min(best, {w[t] - mat[t][t], co[t]});
13  co[s].insert(co[s].end(), all(co[t]));
14  rep(i,0,n) mat[s][i] += mat[t][i];
15  rep(i,0,n) mat[i][s] = mat[s][i];
16  mat[0][t] = INT_MIN;
17  }
18  return best;
19}
```

5.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: vi btoa(m, -1); hopcroftKarp(g, btoa);

Time: $\mathcal{O}(\sqrt{VE})$

f612e4, 39 lines

```
1 bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
2   if (A[a] != L) return 0;
3   A[a] = -1;
4   for (int b : g[a]) if (B[b] == L + 1) {
5       B[b] = 0;
6       if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
7           return btoa[b] = a, 1;
8   }
9   return 0;
10 }
11
12 int hopcroftKarp(vector<vi>& g, vi& btoa) {
13     int res = 0;
14     vi A(g.size()), B(btoa.size()), cur, next;
15     for (;;) {
16         fill(all(A), 0); fill(all(B), 0);
17         cur.clear();
18         for (int a : btoa) if (a != -1) A[a] = -1;
19         rep(a,0,sz(g)) if (A[a] == 0) cur.push_back(a);
20         for (int lay = 1;; lay++) {
21             bool islast = 0;
22             next.clear();
23             for (int a : cur) for (int b : g[a]) {
24                 if (btoa[b] == -1) {
25                     B[b] = lay;
26                     islast = 1;
27                 }
28                 else if (btoa[b] != a && !B[b]) {
29                     B[b] = lay;
30                     next.push_back(btoa[b]);
31                 }
32             }
33             if (islast) break;
34             if (next.empty()) return res;
35             for (int a : next) A[a] = lay;
36             cur.swap(next);
37         }
38         rep(a,0,sz(g)) res += dfs(a, 0, g, btoa, A, B);
39     }
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: vi btoa(m, -1); dfsMatching(g, btoa);

Time: $\mathcal{O}(VE)$

522b98, 22 lines

```
1 bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
```

```
2   if (btoa[j] == -1) return 1;
3   vis[j] = 1; int di = btoa[j];
4   for (int e : g[di])
5       if (!vis[e] && find(e, g, btoa, vis)) {
6           btoa[e] = di;
7           return 1;
8       }
9   return 0;
10 }
11 int dfsMatching(vector<vi>& g, vi& btoa) {
12     vi vis;
13     rep(i,0,sz(g)) {
14         vis.assign(sz(btoa), 0);
15         for (int j : g[i])
16             if (find(j, g, btoa, vis)) {
17                 btoa[j] = i;
18                 break;
19             }
20     }
21     return sz(btoa) - (int)count(all(btoa), -1);
22 }
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

DFSMatching.h

da4196, 20 lines

```
1 vi cover(vector<vi>& g, int n, int m) {
2     vi match(m, -1);
3     int res = dfsMatching(g, match);
4     vector<bool> lfound(n, true), seen(m);
5     for (int it : match) if (it != -1) lfound[it] = false;
6     vi q, cover;
7     rep(i,0,n) if (lfound[i]) q.push_back(i);
8     while (!q.empty()) {
9         int i = q.back(); q.pop_back();
10        lfound[i] = 1;
11        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
12            seen[e] = true;
13            q.push_back(match[e]);
14        }
15    }
16    rep(i,0,n) if (!lfound[i]) cover.push_back(i);
17    rep(i,0,m) if (seen[i]) cover.push_back(n+i);
18    assert(sz(cover) == res);
19    return cover;
20 }
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j]$ = cost for $L[i]$ to be matched with $R[j]$ and returns (min cost, match), where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}(N^2M)$

1e0fe9, 31 lines

```
1 pair<int, vi> hungarian(const vector<vi> &a) {
2     if (a.empty()) return {0, {}};
3     int n = sz(a) + 1, m = sz(a[0]) + 1;
4     vi u(n), v(m), p(m), ans(n - 1);
5     rep(i,1,n) {
6         p[0] = i;
7         int j0 = 0; // add "dummy" worker 0
8         vi dist(m, INT_MAX), pre(m, -1);
9         vector<bool> done(m + 1);
10        do { // dijkstra
11            done[j0] = true;
12            int i0 = p[j0], j1, delta = INT_MAX;
13            rep(j,1,m) if (!done[j]) {
14                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
15                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
16                if (dist[j] < delta) delta = dist[j], j1 = j;
17            }
18            rep(j,0,m) {
19                if (done[j]) u[p[j]] += delta, v[j] -= delta;
20                else dist[j] -= delta;
```

```
21        }
22        j0 = j1;
23    } while (p[j0]);
24    while (j0) { // update alternating path
25        int j1 = pre[j0];
26        p[j0] = p[j1], j0 = j1;
27    }
28 }
29 rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
30 return {-v[0], ans}; // min cost
31 }
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod .

Time: $\mathcal{O}(N^3)$

MatrixInverse-mod.h

cb1912, 40 lines

```
1 vector<pii> generalMatching(int N, vector<pii>& ed) {
2     vector<vector<ll>> mat(N, vector<ll>(N)), A;
3     for (pii pa : ed) {
4         int a = pa.first, b = pa.second, r = rand() % mod;
5         mat[a][b] = r, mat[b][a] = (mod - r) % mod;
6     }
7
8     int r = matInv(A = mat), M = 2*N - r, fi, fj;
9     assert(r % 2 == 0);
10
11    if (M != N) do {
12        mat.resize(M, vector<ll>(M));
13        rep(i,0,N) {
14            mat[i].resize(M);
15            rep(j,N,M) {
16                int r = rand() % mod;
17                mat[i][j] = r, mat[j][i] = (mod - r) % mod;
18            }
19        }
20    } while (matInv(A = mat) != M);
21
22    vi has(M, 1); vector<pii> ret;
23    rep(it,0,M/2) {
24        rep(i,0,M) if (has[i])
25            rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
26                fi = i; fj = j; goto done;
27            }
28        assert(0); done:
29        if (fj < N) ret.emplace_back(fi, fj);
30        has[fi] = has[fj] = 0;
31        rep(sw,0,2) {
32            ll a = modpow(A[fi][fj], mod-2);
33            rep(i,0,M) if (has[i] && A[i][fj]) {
34                ll b = A[i][fj] * a % mod;
35                rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
36            }
37            swap(fi, fj);
38        }
39        return ret;
40    }
```

5.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.

Time: $\mathcal{O}(E + V)$

76b5c9, 24 lines

```
1 vi val, comp, z, cont;
2 int Time, ncomps;
3 template<class G, class F> int dfs(int j, G& g, F& f) {
4     int low = val[j] = ++Time, x; z.push_back(j);
5     for (auto e : g[j]) if (comp[e] < 0)
```

```
6 low = min(low, val[e] ?: dfs(e,g,f));
7
8 if (low == val[j]) {
9     do {
10         x = z.back(); z.pop_back();
11         comp[x] = ncomps;
12         cont.push_back(x);
13     } while (x != j);
14     f(cont); cont.clear();
15     ncomps++;
16 }
17 return val[j] = low;
18}
19template<class G, class F> void scc(G& g, F f) {
20    int n = sz(g);
21    val.assign(n, 0); comp.assign(n, -1);
22    Time = ncomps = 0;
23    rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
24}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps[&](const vi& edgelist) {...};
Time: $\mathcal{O}(E + V)$

c6b7c7, 32 lines

```
1vi num, st;
2vector<vector<pii>> ed;
3int Time;
4template<class F>
5int dfs(int at, int par, F& f) {
6    int me = num[at] = ++Time, top = me;
7    for (auto [y, e] : ed[at]) if (e != par) {
8        if (num[y]) {
9            top = min(top, num[y]);
10           if (num[y] < me)
11               st.push_back(e);
12         } else {
13             int si = sz(st);
14             int up = dfs(y, e, f);
15             top = min(top, up);
16             if (up == me) {
17                 st.push_back(e);
18                 f(vi(st.begin() + si, st.end()));
19                 st.resize(si);
20             }
21             else if (up < me) st.push_back(e);
22             else /* e is a bridge */
23                 }
24         }
25     return top;
26}
27
28template<class F>
29void bicomps(F f) {
30    num.assign(sz(ed), 0);
31    rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
32}
```

2SAT.h

Description: Calculates a valid assignment to boolean variables $a, b, c, \dots$ to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\dots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions ($\sim x$).

Usage: TwoSAT ts(# of bool vars);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable

ts.values[0..N-1] holds the assigned values to the vars

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

fd8744, 46 lines

```
1struct TwoSAT {
2    int N; vector<vi> gr;
3    vi values; // 0 = false, 1 = true
4    TwoSAT(int n = 0) : N(n), gr(2*n) {}
5    int addVar() { // (optional)
6        gr.emplace_back(); gr.emplace_back(); return N++;
7    }
8
9    void either(int f, int j) {
10        f = max(2*f, -1-2*f); j = max(2*j, -1-2*j);
11        gr[f].push_back(j^1); gr[j].push_back(f^1);
12    }
13    void setValue(int x) { either(x, x); }
14
15    void atMostOne(const vi& li) { // (optional)
16        if (sz(li) <= 1) return;
17        int cur = ~li[0];
18        rep(i,2,sz(li)) {
19            int next = addVar(); either(cur, ~li[i]);
20            either(cur, next); either(~li[i], next);
21            cur = ~next;
22        }
23        either(cur, ~li[1]);
24    }
25
26    vi val, comp, z; int time = 0;
27    int dfs(int i) {
28        int low = val[i] = ++time, x; z.push_back(i);
29        for(int e : gr[i]) if (!comp[e])
30            low = min(low, val[e] ?: dfs(e));
31        if (low == val[i]) do {
32            x = z.back(); z.pop_back(); comp[x] = low;
33            if (values[x]>1] == -1)
34                values[x]>1] = x&1;
35        } while (x != i);
36        return val[i] = low;
37    }
38
39    bool solve() {
40        values.assign(N, -1);
41        val.assign(2*N, 0); comp = val;
42        rep(i,0,2*N) if (!comp[i]) dfs(i);
43        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
44        return 1;
45    }
46};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

780b64, 15 lines

```
1vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
2    int n = sz(gr);
3    vi D(n), its(n), eu(nedges), ret, s = {src};
4    D[src]++; // to allow Euler paths, not just cycles
5    while (!s.empty()) {
6        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
7        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
8        tie(y, e) = gr[x][it++];
9        if (!eu[e]) {
10            D[x]--, D[y]++;
11            eu[e] = 1; s.push_back(y);
12        }
13    }
14    for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
15    return {ret.rbegin(), ret.rend()};
16}
```

5.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
1vi edgeColoring(int N, vector<pii> eds) {
2    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
3    for (pii e : eds) ++cc[e.first], ++cc[e.second];
4    int u, v, ncols = *max_element(all(cc)) + 1;
5    vector<vi> adj(N, vi(ncols, -1));
6    for (pii e : eds) {
7        tie(u, v) = e;
8        fan[0] = v;
9        loc.assign(ncols, 0);
10       vi at = u, end = u, d, c = free[u], ind = 0, i = 0;
11       while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
12           loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
13       cc[loc[d]] = c;
14       for (int cd = d; at != -1; cd ^= c ^ at, at = adj[at][cd])
15           swap(adj[at][cd], adj[end = at][cd ^ c ^ at]);
16       while (adj[fan[i]][d] != -1) {
17           int left = fan[i], right = fan[++i], e = cc[i];
18           adj[u][e] = left;
19           adj[left][e] = u;
20           adj[right][e] = -1;
21           free[right] = e;
22       }
23       adj[u][d] = fan[i];
24       adj[fan[i]][d] = u;
25       for (int y : {fan[0], u, end})
26           for (int& z = free[y] = 0; adj[y][z] != -1; z++);
27     }
28     rep(i,0,sz(eds))
29         for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
30     return ret;
31}
```

5.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}(3^{n/3})$, much faster for sparse graphs

b0d5b1, 12 lines

```
1typedef bitset<128> B;
2template<class F>
3void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
4    if (!P.any()) { if (!X.any()) f(R); return; }
5    auto q = (P | X)._Find_first();
6    auto cand = P & ~eds[q];
7    rep(i,0,sz(eds)) if (cand[i]) {
8        R[i] = 1;
9        cliques(eds, f, P & eds[i], X & eds[i], R);
10       R[i] = P[i] = 0; X[i] = 1;
11     }
12}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
1typedef vector<bitset<200>> vb;
2struct Maxclique {
3    double limit=0.025, pk=0;
4    struct Vertex { int i, d=0; };
5    typedef vector<Vertex> vv;
```



```

6 vb e;
7 vv V;
8 vector<vi> C;
9 vi qmax, q, S, old;
10 void init(vv& r) {
11     for (auto& v : r) v.d = 0;
12     for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
13     sort(all(r), [](auto a, auto b) { return a.d > b.d; });
14     int mxD = r[0].d;
15     rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
16 }
17 void expand(vv& R, int lev = 1) {
18     S[lev] += S[lev - 1] - old[lev];
19     old[lev] = S[lev - 1];
20     while (sz(R)) {
21         if (sz(q) + R.back().d <= sz(qmax)) return;
22         q.push_back(R.back().i);
23         vv T;
24         for (auto v:R) if (e[R.back().i][v.i] T.push_back({v.i});
25         if (sz(T)) {
26             if (S[lev]++ / ++pk < limit) init(T);
27             int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
28             C[1].clear(), C[2].clear();
29             for (auto v : T) {
30                 int k = 1;
31                 auto f = [&](int i) { return e[v.i][i]; };
32                 while (any_of(all(C[k]), f)) k++;
33                 if (k > mxk) mxk = k, C[mxk + 1].clear();
34                 if (k < mnk) T[j++] .i = v.i;
35                 C[k].push_back(v.i);
36             }
37             if (j > 0) T[j - 1].d = 0;
38             rep(k,mnk,mxk + 1) for (int i : C[k])
39                 T[j].i = i, T[j++].d = k;
40             expand(T, lev + 1);
41         } else if (sz(q) > sz(qmax)) qmax = q;
42         q.pop_back(), R.pop_back();
43     }
44 }
45 vi maxClique() { init(V), expand(V); return qmax; }
46 MaxClique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
47     rep(i,0,sz(e)) V.push_back({i});
48 }
49};

```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

5.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```

1 vector<vi> treeJump(vi& P) {
2     int on = 1, d = 1;
3     while (on < sz(P)) on *= 2, d++;
4     vector<vi> jmp(d, P);
5     rep(i,1,d) rep(j,0,sz(P))
6         jmp[i][j] = jmp[i-1][jmp[i-1][j]];
7     return jmp;
8 }
9
10 int jmp(vector<vi>& tbl, int nod, int steps) {
11     rep(i,0,sz(tbl))
12         if (steps < (1<<i)) nod = tbl[i][nod];
13     return nod;
14 }
15
16 int lca(vector<vi>& tbl, vi& depth, int a, int b) {
17     if (depth[a] < depth[b]) swap(a, b);
18     a = jmp(tbl, a, depth[a] - depth[b]);
19     if (a == b) return a;
20     for (int i = sz(tbl); i--;) {
21         int c = tbl[i][a], d = tbl[i][b];

```

```

22     if (c != d) a = c, b = d;
23 }
24 return tbl[0][a];
25 }

```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

RMQ.h

0f62fb, 21 lines

```

1 struct LCA {
2     int T = 0;
3     vi time, path, ret;
4     RMQ<int> rmq;
5
6     LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
7     void dfs(vector<vi>& C, int v, int par) {
8         time[v] = T++;
9         for (int y : C[v]) if (y != par) {
10             path.push_back(v), ret.push_back(time[v]);
11             dfs(C, y, v);
12         }
13     }
14
15     int lca(int a, int b) {
16         if (a == b) return a;
17         tie(a, b) = minmax(time[a], time[b]);
18         return path[rmq.query(a, b)];
19     }
20     //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
21 };

```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

LCA.h

9775a0, 21 lines

```

1 typedef vector<pair<int, int>> vpi;
2 vpi compressTree(LCA& lca, const vi& subset) {
3     static vi rev; rev.resize(sz(lca.time));
4     vi li = subset, &T = lca.time;
5     auto cmp = [&](int a, int b) { return T[a] < T[b]; };
6     sort(all(li), cmp);
7     int m = sz(li)-1;
8     rep(i,0,m) {
9         int a = li[i], b = li[i+1];
10        li.push_back(lca.lca(a, b));
11    }
12    sort(all(li), cmp);
13    li.erase(unique(all(li), li.end()));
14    rep(i,0,sz(li)) rev[li[i]] = i;
15    vpi ret = {pii(0, li[0])};
16    rep(i,0,sz(li)-1) {
17        int a = li[i], b = li[i+1];
18        ret.emplace_back(rev[lca.lca(a, b)], b);
19    }
20    return ret;
21 }

```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

0fb462, 90 lines

```

1 struct Node { // Splay tree. Root's pp contains tree's parent.
2     Node *p = 0, *pp = 0, *c[2];
3     bool flip = 0;
4     Node() { c[0] = c[1] = 0; fix(); }
5     void fix() {
6         if (c[0]) c[0]->p = this;
7         if (c[1]) c[1]->p = this;

```

```

8     // (+ update sum of subtree elements etc. if wanted)
9 }
10 void pushFlip() {
11     if (!flip) return;
12     flip = 0; swap(c[0], c[1]);
13     if (c[0]) c[0]->flip ^= 1;
14     if (c[1]) c[1]->flip ^= 1;
15 }
16 int up() { return p ? p->c[1] == this ? -1; }
17 void rot(int i, int b) {
18     int h = i ^ b;
19     Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
20     if ((y->p = p)) p->c[up()] = y;
21     c[i] = z->c[i ^ 1];
22     if (b < 2) {
23         x->c[h] = y->c[h ^ 1];
24         y->c[h ^ 1] = x;
25     }
26     z->c[i ^ 1] = this;
27     fix(); x->fix(); y->fix();
28     if (p) p->fix();
29     swap(pp, y->pp);
30 }
31 void splay() {
32     for (pushFlip(); p; ) {
33         if (p->p) p->p->pushFlip();
34         p->pushFlip(); pushFlip();
35         int c1 = up(), c2 = p->up();
36         if (c2 == -1) p->rot(c1, 2);
37         else p->p->rot(c2, c1 != c2);
38     }
39 }
40 Node* first() {
41     pushFlip();
42     return c[0] ? c[0]->first() : (splay(), this);
43 }
44 };
45
46 struct LinkCut {
47     vector<Node> node;
48     LinkCut(int N) : node(N) {}
49
50     void link(int u, int v) { // add an edge (u, v)
51         assert(!connected(u, v));
52         makeRoot(&node[u]);
53         node[u].pp = &node[v];
54     }
55     void cut(int u, int v) { // remove an edge (u, v)
56         Node *x = &node[u], *top = &node[v];
57         makeRoot(top); x->splay();
58         assert(top == (x->pp ? x->c[0]));
59         if (x->pp) x->pp = 0;
60         else {
61             x->c[0] = top->p = 0;
62             x->fix();
63         }
64     }
65     bool connected(int u, int v) { // are u, v in the same tree?
66         Node* nu = access(&node[u])->first();
67         return nu == access(&node[v])->first();
68     }
69     void makeRoot(Node* u) {
70         access(u);
71         u->splay();
72         if (u->c[0]) {
73             u->c[0]->p = 0;
74             u->c[0]->flip ^= 1;
75             u->c[0]->pp = u;
76             u->c[0] = 0;
77             u->fix();
78         }
79     }
80     Node* access(Node* u) {
81         u->splay();
82         while (Node* pp = u->pp) {
83             pp->splay(); u->pp = 0;
84             if (pp->c[1]) {

```

```
85     pp->c[l]->p = 0; pp->c[l]->pp = pp; }
86     pp->c[l] = u; pp->fix(); u = pp;
87 }
88 return u;
89 }
90};
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

Time: $\mathcal{O}((\log N)^2)$
KactLLazySegmentTree.h 9547af, 46 lines

```
1 template <bool VALS_EDGES> struct HLD {
2     int N, tim = 0;
3     vector<vi> adj;
4     vi par, siz, rt, pos;
5     Node *tree;
6     HLD(vector<vi> adj_)
7     : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
8       rt(N), pos(N), tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
9     void dfsSz(int v) {
10         for (int& u : adj[v]) {
11             adj[u].erase(find(all(adj[u]), v));
12             par[u] = v;
13             dfsSz(u);
14             siz[v] += siz[u];
15             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
16         }
17     }
18     void dfsHld(int v) {
19         pos[v] = tim++;
20         for (int u : adj[v]) {
21             rt[u] = (u == adj[v][0] ? rt[v] : u);
22             dfsHld(u);
23         }
24     }
25     template <class B> void process(int u, int v, B op) {
26         for (; v = par[rt[v]]) {
27             if (pos[u] > pos[v]) swap(u, v);
28             if (rt[u] == rt[v]) break;
29             op(pos[rt[v]], pos[v] + 1);
30         }
31         op(pos[u] + VALS_EDGES, pos[v] + 1);
32     }
33     void modifyPath(int u, int v, int val) {
34         process(u, v, [&](int l, int r) { tree->add(l, r, val); });
35     }
36     int queryPath(int u, int v) { // Modify depending on problem
37         int res = -1e9;
38         process(u, v, [&](int l, int r) {
39             res = max(res, tree->query(l, r));
40         });
41         return res;
42     }
43     int querySubtree(int v) { // modifySubtree is similar
44         return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
45     }
46};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$
UnionFindRollback.h 39e620, 57 lines

```
1 struct Edge { int a, b; ll w; };
2 struct Node {
3     Edge key; Node *l, *r; ll delta;
4     void prop() {
5         key.w += delta;
6         if (l) l->delta += delta;
7         if (r) r->delta += delta;
```

```
8     delta = 0;
9     }
10 Edge top() { prop(); return key; }
11};
12 Node *merge(Node *a, Node *b) {
13     if (!a || !b) return a ?: b;
14     a->prop(), b->prop();
15     if (a->key.w > b->key.w) swap(a, b);
16     swap(a->l, (a->r = merge(b, a->r)));
17     return a;
18 }
19 void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
20
21 pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
22     RollbackUF uf(n); vector<Node*> heap(n);
23     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
24     ll res = 0;
25     vi seen(n, -1), path(n), par(n);
26     seen[r] = r;
27     vector<Edge> Q(n); in(n, {-1,-1}), comp;
28     deque<tuple<int, int, vector<Edge>>> cys;
29     rep(s,0,n) {
30         int u = s, qi = 0, w;
31         while (seen[u] < 0) {
32             if (!heap[u]) return {-1,{};};
33             Edge e = heap[u]->top();
34             heap[u]->delta -= e.w, pop(heap[u]);
35             Q[qi] = e, path[qi++] = u, seen[u] = s;
36             res += e.w, u = uf.find(e.a);
37             if (seen[u] == s) {
38                 Node* cyc = 0;
39                 int end = qi, time = uf.time();
40                 do cyc = merge(cyc, heap[w = path[--qi]]);
41                 while (uf.join(u, w));
42                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
43                 cys.push_front({u, time, {&Q[qi], &Q[end]} });
44             }
45         }
46         rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
47     }
48
49     for (auto& [u,t,comp] : cys) { // restore sol (optional)
50         uf.rollback(t);
51         Edge inEdge = in[u];
52         for (auto& e : comp) in[uf.find(e.b)] = e;
53         in[uf.find(inEdge.b)] = inEdge;
54     }
55     rep(i,0,n) par[i] = in[i].a;
56     return {res, par};
57 }
```

5.8 Math

5.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat, and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

5.8.2 Erdős-Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (6)

6.1 Geometric primitives

Point.h

Description: Class to handle points in $\mathbb{R}^2$. T can be e.g. double or long long.

```
1 template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
2 template<class T>
```

```
3 struct Point {
4     typedef Point P;
5     T x, y;
6     explicit Point(T x=0, T y=0) : x(x), y(y) {}
7     bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
8     bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
9     P operator+(P p) const { return P(x+p.x, y+p.y); }
10    P operator-(P p) const { return P(x-p.x, y-p.y); }
11    P operator*(T d) const { return P(x*d, y*d); }
12    P operator/(T d) const { return P(x/d, y/d); }
13    T dot(P p) const { return x*p.x + y*p.y; }
14    T cross(P p) const { return x*p.y - y*p.x; }
15    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
16    T dist2() const { return x*x + y*y; }
17    double dist() const { return sqrt((double)dist2()); }
18    // angle to x-axis in interval [-pi, pi]
19    double angle() const { return atan2(y, x); }
20    P unit() const { return *this/dist(); } // makes dist()==1
21    P perp() const { return P(-y, x); } // rotates +90 degrees
22    P normal() const { return perp().unit(); }
23    // returns point rotated 'a' radians ccw around the origin
24    P rotate(double a) const {
25        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
26    friend ostream& operator<<(ostream& os, P p) {
27        return os << "(" << p.x << ", " << p.y << ")"; }
28};
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

Point.h c597e8, 3 lines

```
1 template<class P> bool onSegment(P s, P e, P p) {
2     return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
3 }
```

SideOf.h

Description: Returns where p is as seen from s towards e . $1/0/-1 \Leftrightarrow$ left/on line/right. If the optional argument ϵ is given 0 is returned if p is within distance ϵ from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;
Point.h 3af81c, 9 lines

```
1 template<class P>
2 int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
3
4 template<class P>
5 int sideOf(const P& s, const P& e, const P& p, double eps) {
6     auto a = (e-s).cross(p-s);
7     double l = (e-s).dist()*eps;
8     return (a > l) - (a < -l);
9 }
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.

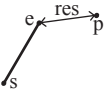
Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

Point.h 5c88f4, 3 lines

```
1 typedef Point<double> P;
2 double segDist(P& s, P& e, P& p) {
3     if (s==e) return (p-s).dist();
4     auto d = (e-s).dist2(), t = min(d,max(.,(p-s).dot(e-s)));
5     return ((p-s)*d-(e-s)*t).dist()/d;
6 }
```

SegmentIntersection.h

Description:



If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1) cout << "intersect at " << inter[0] << endl;
Point.h, OnSegment.h9d57f2, 13 lines

```
1 template<class P> vector<P> segInter(P a, P b, P c, P d) {
2     auto oa = c.cross(d, a), ob = c.cross(d, b),
3       oc = a.cross(b, c), od = a.cross(b, d);
4     // Checks if intersection is single non-endpoint point.
5     if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
6         return {(a * ob - b * oa) / (ob - oa)};
7     set<P> s;
8     if (onSegment(c, d, a)) s.insert(a);
9     if (onSegment(c, d, b)) s.insert(b);
10    if (onSegment(a, b, c)) s.insert(c);
11    if (onSegment(a, b, d)) s.insert(d);
12    return {all(s)};
13 }
```

LineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

Point.hf6bf6b, 4 lines

```
1 template<class P>
2 double lineDist(const P& a, const P& b, const P& p) {
3     return (double) (b-a).cross(p-a)/(b-a).dist();
4 }
```

LineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

Usage: auto res = lineInter(s1,e1,s2,e2);
if(res.first == 1) cout << "intersect at " << res.second << endl;
Point.ha01f81, 8 lines

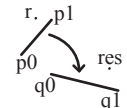
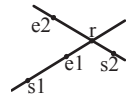
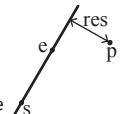
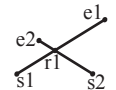
```
1 template<class P>
2 pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
3     auto d = (e1 - s1).cross(e2 - s2);
4     if (d == 0) // if parallel
5         return {-(s1.cross(e1, s2) == 0), P(0, 0)};
6     auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
7     return {1, (s1 * p + e1 * q) / d};
8 }
```

LinearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

Point.h03a306, 6 lines

```
1 typedef Point<double> P;
2 P linearTransformation(const P& p0, const P& p1,
3   const P& q0, const P& q1, const P& r) {
4     P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
```



```
5     return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
6 }
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j;
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

0f0602, 35 lines

```
1 struct Angle {
2     int x, y;
3     int t;
4     Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
5     Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
6     int half() const {
7         assert(x || y);
8         return y < 0 || (y == 0 && x < 0);
9     }
10    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
11    Angle t180() const { return {-x, -y, t + half(); }
12    Angle t360() const { return {x, y, t + 1}; }
13 };
14 bool operator<(Angle a, Angle b) {
15     // add a.dist2() and b.dist2() to also compare distances
16     return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
17            make_tuple(b.t, b.half(), a.x * (ll)b.y);
18 }
19
20 // Given two points, this calculates the smallest angle between
21 // them, i.e., the angle that covers the defined line segment.
22 pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
23     if (b < a) swap(a, b);
24     return (b < a.t180() ?
25            make_pair(a, b) : make_pair(b, a.t360()));
26 }
27 Angle operator+(Angle a, Angle b) { // point a + vector b
28     Angle r(a.x + b.x, a.y + b.y, a.t);
29     if (a.t180() < r) r.t--;
30     return r.t180() < a ? r.t360() : r;
31 }
32 Angle angleDiff(Angle a, Angle b) { // angle b - angle a
33     int tu = b.t - a.t; a.t = b.t;
34     return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
35 }
```

6.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

Point.h84d6d3, 11 lines

```
1 typedef Point<double> P;
2 bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
3     if (a == b) { assert(r1 != r2); return false; }
4     P vec = b - a;
5     double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
6           p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
7     if (sum*sum < d2 || dif*dif > d2) return false;
8     P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
9     *out = {mid + per, mid - per};
10    return true;
11 }
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

Point.hb0153d, 13 lines

```
1 template<class P>
2 vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
3     P d = c2 - c1;
4     double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
5     if (d2 == 0 || h2 < 0) return {};
6     vector<pair<P, P>> out;
7     for (double sign : {-1, 1}) {
8         P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
9         out.push_back({c1 + v * r1, c2 + v * r2});
10    }
11    if (h2 == 0) out.pop_back();
12    return out;
13 }
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

Point.h e0cfba, 9 lines

```
1 template<class P>
2 vector<P> circleLine(P c, double r, P a, P b) {
3     P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
4     double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
5     if (h2 < 0) return {};
6     if (h2 == 0) return {p};
7     P h = ab.unit() * sqrt(h2);
8     return {p - h, p + h};
9 }
```

CirclePolygonIntersection.h

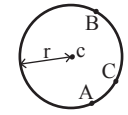
Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$
Point.ha1ee63, 19 lines

```
1 typedef Point<double> P;
2 #define arg(p, q) atan2(p.cross(q), p.dot(q))
3 double circlePoly(P c, double r, vector<P> ps) {
4     auto tri = [&](P p, P q) {
5         auto r2 = r * r / 2;
6         P d = q - p;
7         auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
8         auto det = a * a - b;
9         if (det <= 0) return arg(p, q) * r2;
10        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
11        if (t < 0 || 1 <= s) return arg(p, q) * r2;
12        P u = p + d * s, v = p + d * t;
13        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
14    };
15    auto sum = 0.0;
16    rep(i,0,sz(ps))
17        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
18    return sum;
19 }
```

Circumcircle.h

Description:
The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



Point.h1caa3a, 9 lines

```
1 typedef Point<double> P;
2 double ccRadius(const P& A, const P& B, const P& C) {
3     return (B-A).dist()*(C-B).dist()*(A-C).dist()/
4            abs((B-A).cross(C-A))/2;
5 }
6 P ccCenter(const P& A, const P& B, const P& C) {
7     P b = C-A, c = B-A;
8     return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
9 }
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

Circumcircle.h09dd0a, 15 lines

```
1pair<P, double> mec(vector<P> ps) {
2  shuffle(all(ps), mt19937(time(0))));
3  P o = ps[0];
4  double r = 0, EPS = 1 + 1e-8;
5  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
6    o = ps[i], r = 0;
7    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
8      o = (ps[i] + ps[j]) / 2;
9      r = (o - ps[i]).dist();
10     rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
11       o = ccCenter(ps[i], ps[j], ps[k]);
12       r = (o - ps[i]).dist();
13     } } }
14  return {o, r};
15}
```

6.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$
Point.h, OnSegment.h, SegmentDistance.h2bf504, 11 lines

```
1template<class P>
2bool inPolygon(vector<P> &p, P a, bool strict = true) {
3  int cnt = 0, n = sz(p);
4  rep(i,0,n) {
5    P q = p[(i + 1) % n];
6    if (onSegment(p[i], q, a)) return !strict;
7    //or: if (segDist(p[i], q, a) <= eps) return !strict;
8    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
9  }
10  return cnt;
11}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

Point.hf12300, 6 lines

```
1template<class T>
2T polygonArea2(vector<Point<T>>& v) {
3  T a = v.back().cross(v[0]);
4  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
5  return a;
6}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$
Point.h9706dc, 9 lines

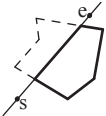
```
1typedef Point<double> P;
2P polygonCenter(const vector<P>& v) {
3  P res(0, 0); double A = 0;
4  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
5    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
6    A += v[j].cross(v[i]);
7  }
8  return res / A / 3;
9}
```

PolygonCut.h

Description: Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

Point.h, LineIntersection.hf2b7d4, 13 lines



```
1typedef Point<double> P;
2vector<P> polygonCut(const vector<P>& poly, P s, P e) {
3  vector<P> res;
4  rep(i,0,sz(poly)) {
5    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
6    bool side = s.cross(e, cur) < 0;
7    if (side != (s.cross(e, prev) < 0))
8      res.push_back(lineInter(s, e, cur, prev).second);
9    if (side)
10     res.push_back(cur);
11  }
12  return res;
13}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

Time: $\mathcal{O}(N^2)$, where N is the total number of points
Point.h, sideOf.h3931c6, 31 lines

```
1typedef Point<double> P;
2double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
3double polyUnion(vector<vector<P>>& poly) {
4  double ret = 0;
5  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
6    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
7    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
8    rep(j,0,sz(poly)) if (i != j) {
9      rep(u,0,sz(poly[j])) {
10       P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
11       int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
12       if (sc != sd) {
13         double sa = C.cross(D, A), sb = C.cross(D, B);
14         if (min(sc, sd) < 0)
15           segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
16       } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
17         segs.emplace_back(rat(C - A, B - A), 1);
18         segs.emplace_back(rat(D - A, B - A), -1);
19       } }
20    sort(all(segs));
21    for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
22    double sum = 0;
23    int cnt = segs[0].second;
24    rep(j,1,sz(segs)) {
25      if (!cnt) sum += segs[j].first - segs[j - 1].first;
26      cnt += segs[j].second;
27    }
28    ret += A.cross(B) * sum;
29  }
30  return ret / 2;
31}
```

ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$
Point.h310954, 13 lines

```
1typedef Point<ll> P;
2vector<P> convexHull(vector<P> pts) {
3  if (sz(pts) <= 1) return pts;
4  sort(all(pts));
5  vector<P> h(sz(pts)+1);
6  int s = 0, t = 0;
7  for (int it = 2; it--; s = --t, reverse(all(pts)))
8    for (P p : pts) {
9      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
10     h[t++] = p;
11  }
12  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
13}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no



duplicate/collinear points).

Time: $\mathcal{O}(n)$
Point.hc571b8, 12 lines

```
1typedef Point<ll> P;
2array<P, 2> hullDiameter(vector<P> S) {
3  int n = sz(S), j = n < 2 ? 0 : 1;
4  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
5  rep(i,0,j)
6    for (; j = (j + 1) % n) {
7      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
8      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
9        break;
10    }
11  return res.second;
12}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$
Point.h, SideOf.h, OnSegment.h71446b, 14 lines

```
1typedef Point<ll> P;
2
3bool inHull(const vector<P>& l, P p, bool strict = true) {
4  int a = 1, b = sz(l) - 1, r = !strict;
5  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
6  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
7  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
8    return false;
9  while (abs(a - b) > 1) {
10   int c = (a + b) / 2;
11   (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
12 }
13 return sgn(l[a].cross(l[b], p)) < r;
14}
```

HalfPlaneIntersection.h

Description: Intersect left half planes defined by s and t . Consider adding a bounding box. Hull may degenerate to a point.

Time: $\mathcal{O}(n \log n)$
Point.h32d1d2, 43 lines

```
1const double eps = 1e-9;
2int sgn(double x) { return (x > eps) - (x < -eps); }
3template<class P>
4struct L {
5  P s, t;
6  double a;
7  explicit L(P s=P(), P t=P()) : s(s), t(t), a((t-s).angle()) {}
8  bool operator==(const L &l) const { return sgn(a-l.a) == 0; }
9  bool operator<(const L &l) const {
10   if (sgn(a-l.a) == 0) return sgn((t-s).cross(l.s-s)) < 0;
11   return a < l.a;
12 }
13 bool out(const P &p) const { // check if outside HP
14   return sgn((t-s).cross(p-s)/(t-s).dist()) < 0; // < 1 for
15   ⇐ degen case
16 }
17 P inter(const L &l) const { // assume never parallel
18   auto d = (t-s).cross(l.t-l.s); // d != 0
19   auto p = l.s.cross(t, l.t), q = l.s.cross(l.t, s);
20   return (s * p + t * q) / d;
21 }
22
23template<class P>
24vector<P> HPI(vector<L<P>> &l){
25  sort(all(l)); l.erase(unique(all(l)), l.end());
26
27  deque<L<P>> dq; int sz = 0;
28  rep(i, 0, sz(l)) {
```



```

29 while (sz > 1 && l[i].out(dq[sz-1].inter(dq[sz-2])))
30   dq.pop_back(), --sz;
31 while (sz > 1 && l[i].out(dq[0].inter(dq[1])))
32   dq.pop_front(), --sz;
33   dq.push_back(l[i]), ++sz;
34 }
35 while (sz > 2 && dq[0].out(dq[sz-1].inter(dq[sz-2])))
36   dq.pop_back(), --sz;
37 while (sz > 2 && dq[sz-1].out(dq[0].inter(dq[1])))
38   dq.pop_front(), --sz;
39 if (sz < 3) return {};
40 vector<P> res;
41 rep(i, 0, sz) res.push_back(dq[i].inter(dq[(i+1)%sz]));
42 return res;
43 }

```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i+1)$, $\bullet(i, j)$ if crossing sides $(i, i+1)$ and $(j, j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

Point.h 7cf45b, 39 lines

```

1 #define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
2 #define extr(i) cmp(i+1, i) >= 0 && cmp(i, i-1+n) < 0
3 template <class P> int extrVertex(vector<P>& poly, P dir) {
4   int n = sz(poly), lo = 0, hi = n;
5   if (extr(0)) return 0;
6   while (lo + 1 < hi) {
7     int m = (lo + hi) / 2;
8     if (extr(m)) return m;
9     int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
10    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
11  }
12  return lo;
13 }
14
15 #define cmpl(i) sgn(a.cross(poly[i], b))
16 template <class P>
17 array<int, 2> lineHull(P a, P b, vector<P>& poly) {
18   int endA = extrVertex(poly, (a - b).perp());
19   int endB = extrVertex(poly, (b - a).perp());
20   if (cmpl(endA) < 0 || cmpl(endB) > 0)
21     return {-1, -1};
22   array<int, 2> res;
23   rep(i, 0, 2) {
24     int lo = endB, hi = endA, n = sz(poly);
25     while ((lo + 1) % n != hi) {
26       int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
27       (cmpl(m) == cmpl(endB) ? lo : hi) = m;
28     }
29     res[i] = (lo + !cmpl(hi)) % n;
30     swap(endA, endB);
31   }
32   if (res[0] == res[1]) return {res[0], -1};
33   if (!cmpl(res[0]) && !cmpl(res[1]))
34     switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
35     case 0: return {res[0], res[0]};
36     case 2: return {res[1], res[1]};
37   }
38   return res;
39 }

```

ConvexTangent.h

Description: Given convex polygon and a point strictly outside, compute right/left (+1/-1) (relative) tangent. Arbitrary point on tangent line

Time: $\mathcal{O}(\log n)$

Point.h, SideOf.h 998175, 12 lines

```

1 typedef Point<ll> P;
2 int convexTangent(const vector<P>& h, P p, int sign) {
3   int n = sz(h), a = 0, b = n, c;
4   auto ori = [&](int i, int j) {

```

```

5   return sideOf(p, h[i], h[j%n]) == sign; };
6   if (n < 3) return ori(1, 0) ? 1 : 0;
7   while (a+1 < b) {
8     c = (a+b)/2;
9     ((ori(0, c) ? ori(1, 0) : ori(c, c+1)) ? b : a) = c;
10  }
11  return ori(a, b) ? a : b%n;
12 }

```

6.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

Point.h ac41a6, 17 lines

```

1 typedef Point<ll> P;
2 pair<P, P> closest(vector<P> v) {
3   assert(sz(v) > 1);
4   set<P> S;
5   sort(all(v), [](P a, P b) { return a.y < b.y; });
6   pair<ll, pair<P, P>> ret{{LLONG_MAX, {P(), P()}};
7   int j = 0;
8   for (P p : v) {
9     P d{1 + (ll)sqrt(ret.first), 0};
10    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
11    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
12    for (; lo != hi; ++lo)
13      ret = min(ret, {(lo - p).dist2(), {lo, p}});
14    S.insert(p);
15  }
16  return ret.second;
17 }

```

ManhattanMST.h

Description: Given N points, returns up to $4*N$ edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = |p.x - q.x| + |p.y - q.y|$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.

Time: $\mathcal{O}(N \log N)$

Point.h df6f59, 23 lines

```

1 typedef Point<int> P;
2 vector<array<int, 3>> manhattanMST(vector<P> ps) {
3   vi id(sz(ps));
4   iota(all(id), 0);
5   vector<array<int, 3>> edges;
6   rep(k, 0, 4) {
7     sort(all(id), [&](int i, int j) {
8       return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y; });
9     map<int, int> sweep;
10    for (int i : id) {
11      for (auto it = sweep.lower_bound(-ps[i].y);
12           it != sweep.end(); sweep.erase(it++)) {
13        int j = it->second;
14        P d = ps[i] - ps[j];
15        if (d.y > d.x) break;
16        edges.push_back({d.y + d.x, i, j});
17      }
18      sweep[-ps[i].y] = i;
19    }
20    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
21  }
22  return edges;
23 }

```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

Point.h bac5b0, 63 lines

```

1 typedef long long T;
2 typedef Point<T> P;
3 const T INF = numeric_limits<T>::max();
4
5 bool on_x(const P& a, const P& b) { return a.x < b.x; }
6 bool on_y(const P& a, const P& b) { return a.y < b.y; }
7

```

```

8 struct Node {
9   P pt; // if this is a leaf, the single point in it
10   T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
11   Node *first = 0, *second = 0;
12
13   T distance(const P& p) { // min squared distance to a point
14     T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
15     T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
16     return (P(x,y) - p).dist2();
17   }
18
19   Node(vector<P>&& vp) : pt(vp[0]) {
20     for (P p : vp) {
21       x0 = min(x0, p.x); x1 = max(x1, p.x);
22       y0 = min(y0, p.y); y1 = max(y1, p.y);
23     }
24     if (vp.size() > 1) {
25       // split on x if width >= height (not ideal...)
26       sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
27       // divide by taking half the array for each child (not
28       // best performance with many duplicates in the middle)
29       int half = sz(vp)/2;
30       first = new Node({vp.begin(), vp.begin() + half});
31       second = new Node({vp.begin() + half, vp.end()});
32     }
33   }
34 };
35
36 struct KDTree {
37   Node* root;
38   KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}
39
40   pair<T, P> search(Node *node, const P& p) {
41     if (!node->first) {
42       // uncomment if we should not find the point itself:
43       // if (p == node->pt) return {INF, P()};
44       return make_pair((p - node->pt).dist2(), node->pt);
45     }
46
47     Node *f = node->first, *s = node->second;
48     T bfirst = f->distance(p), bsec = s->distance(p);
49     if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);
50
51     // search closest side first, other side if needed
52     auto best = search(f, p);
53     if (bsec < best.first)
54       best = min(best, search(s, p));
55     return best;
56   }
57
58   // find nearest point to a point, and its squared distance
59   // (requires an arbitrary operator< for Point)
60   pair<T, P> nearest(const P& p) {
61     return search(root, p);
62   }
63 };

```

DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.

Time: $\mathcal{O}(n^2)$

Point.h, 3dHull.h c0e7bc, 10 lines

```

1 template<class P, class F>
2 void delaunay(vector<P>& ps, F trfun) {
3   if (sz(ps) == 3) { int d = (ps[0].cross(ps[1], ps[2]) < 0);
4     trfun(0, 1+d, 2-d); }
5   vector<P3> p3;
6   for (P p : ps) p3.emplace_back(p.x, p.y, p.dist2());
7   if (sz(ps) > 3) for(auto t:hull3d(p3)) if ((p3[t.b]-p3[t.a]).
8     cross(p3[t.c]-p3[t.a]).dot(P3(0,0,1)) < 0)
9     trfun(t.a, t.c, t.b);
10 }

```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

Point.h eefdf5, 88 lines

```
1 typedef Point<ll> P;
2 typedef struct Quad* Q;
3 typedef __int128_t lll; // (can be ll if coords are < 2e4)
4 P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point
5
6 struct Quad {
7     Q rot, o; P p = arb; bool mark;
8     P& F() { return r()->p; }
9     Q& r() { return rot->rot; }
10    Q prev() { return rot->o->rot; }
11    Q next() { return r()->prev(); }
12} *H;
13
14 bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
15     lll p2 = p.dist2(), A = a.dist2()-p2,
16     B = b.dist2()-p2, C = c.dist2()-p2;
17     return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
18}
19 Q makeEdge(P orig, P dest) {
20     Q r = H ? H : new Quad{new Quad{new Quad{}}};
21     H = r->o; r->r()->r() = r;
22     rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
23     r->p = orig; r->F() = dest;
24     return r;
25}
26 void splice(Q a, Q b) {
27     swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
28}
29 Q connect(Q a, Q b) {
30     Q q = makeEdge(a->F(), b->p);
31     splice(q, a->next());
32     splice(q->r(), b);
33     return q;
34}
35
36 pair<Q,Q> rec(const vector<P>& s) {
37     if (sz(s) <= 3) {
38         Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
39         if (sz(s) == 2) return { a, a->r() };
40         splice(a->r(), b);
41         auto side = s[0].cross(s[1], s[2]);
42         Q c = side ? connect(b, a) : 0;
43         return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
44     }
45
46 #define H(e) e->F(), e->p
47 #define valid(e) (e->F()).cross(H(base)) > 0)
48 Q A, B, ra, rb;
49 int half = sz(s) / 2;
50 tie(ra, A) = rec({all(s) - half});
51 tie(B, rb) = rec({sz(s) - half + all(s)});
52 while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
53         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
54 Q base = connect(B->r(), A);
55 if (A->p == ra->p) ra = base->r();
56 if (B->p == rb->p) rb = base;
57
58 #define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
59     while (circ(e->dir->F(), H(base), e->F())) { \
60         Q t = e->dir; \
61         splice(e, e->prev()); \
62         splice(e->r(), e->r()->prev()); \
63         e->o = H; H = e; e = t; \
64     }
65 for (;) {
66     DEL(LC, base->r(), o); DEL(RC, base, prev());
67     if (!valid(LC) && !valid(RC)) break;
68     if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
69         base = connect(RC, base->r());
70     else
71         base = connect(base->r(), LC->r());
```

```
72 }
73 return { ra, rb };
74}
75
76 vector<P> triangulate(vector<P> pts) {
77     sort(all(pts)); assert(unique(all(pts)) == pts.end());
78     if (sz(pts) < 2) return {};
79     Q e = rec(pts).first;
80     vector<Q> q = {e};
81     int qi = 0;
82     while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
83 #define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
84     q.push_back(c->r()); c = c->next(); } while (c != e); }
85 ADD; pts.clear();
86 while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
87 return pts;
88}
```

6.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
1 template<class V, class L>
2 double signedPolyVolume(const V& p, const L& trilst) {
3     double v = 0;
4     for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
5     return v / 6;
6}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

8058ae, 52 lines

```
1 template<class T> struct Point3D {
2     typedef Point3D P;
3     typedef const P& R;
4     T x, y, z;
5     explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
6     bool operator<(R p) const {
7         return tie(x, y, z) < tie(p.x, p.y, p.z); }
8     bool operator==(R p) const {
9         return tie(x, y, z) == tie(p.x, p.y, p.z); }
10    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
11    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
12    P operator*(T d) const { return P(x*d, y*d, z*d); }
13    P operator/(T d) const { return P(x/d, y/d, z/d); }
14    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
15    P cross(R p) const {
16        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
17    }
18    T dist2() const { return x*x + y*y + z*z; }
19    double dist() const { return sqrt((double)dist2()); }
20    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
21    double phi() const { return atan2(y, x); }
22    //Zenith angle (latitude) to the z-axis in interval [0, pi]
23    double theta() const { return atan2(sqrt(x*x+y*y),z); }
24    P unit() const { return *this/(T)dist(); } //makes dist()==1
25    //returns unit vector normal to *this and p
26    P normal(P p) const { return cross(p).unit(); }
27    //returns point rotated 'angle' radians ccw around axis
28    P rotate(double angle, P axis) const {
29        double s = sin(angle), c = cos(angle); P u = axis.unit();
30        return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
31    }
32};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

Point3D.h 5b45fc, 44 lines

```
1 typedef Point3D<double> P3;
2
```

```
3 struct PR {
4     void ins(int x) { (a == -1 ? a : b) = x; }
5     void rem(int x) { (a == x ? a : b) = -1; }
6     int cnt() { return (a != -1) + (b != -1); }
7     int a, b;
8 };
9
10 struct F { P3 q; int a, b, c; };
11
12 vector<F> hull3d(const vector<P3>& A) {
13     assert(sz(A) >= 4);
14     vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
15 #define E(x,y) E[f.x][f.y]
16     vector<F> FS;
17     auto mf = [&](int i, int j, int k, int l) {
18         P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
19         if (q.dot(A[l]) > q.dot(A[i])) q = q * -1;
20         F f{q, i, j, k};
21         E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
22         FS.push_back(f);
23     };
24     rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
25         mf(i, j, k, 6 - i - j - k);
26
27     rep(i,4,sz(A)) {
28         rep(j,0,sz(FS)) {
29             F f = FS[j];
30             if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
31                 E(a,b).rem(f.c); E(a,c).rem(f.b); E(b,c).rem(f.a);
32                 swap(FS[j--], FS.back()); FS.pop_back();
33             }
34         }
35         int nw = sz(FS);
36         rep(j,0,nw) {
37             F f = FS[j];
38 #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
39             C(a, b, c); C(a, c, b); C(b, c, a);
40         }
41         for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
42             A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
43         return FS;
44     };
```

SphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8 lines

```
1 double sphericalDistance(double f1, double t1,
2     double f2, double t2, double radius) {
3     double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
4     double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
5     double dz = cos(t2) - cos(t1);
6     double d = sqrt(dx*dx + dy*dy + dz*dz);
7     return radius*2*asin(d/2);
8}
```

Strings (7)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d4375c, 16 lines

```
1 vi pi(const string& s) {
2   vi p(sz(s));
3   rep(i,1,sz(s)) {
4     int g = p[i-1];
5     while (g && s[i] != s[g]) g = p[g-1];
6     p[i] = g + (s[i] == s[g]);
7   }
8   return p;
9 }
10
11 vi match(const string& s, const string& pat) {
12   vi p = pi(pat + '\0' + s), res;
13   rep(i,sz(p)-sz(s),sz(p))
14     if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
15   return res;
16 }
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

f71b39, 10 lines

```
1 vi Z(const string& s) {
2   vi z(sz(s));
3   int l = -1, r = -1;
4   rep(i,1,sz(s)) {
5     z[i] = i >= r ? 0 : min(r - i, z[i - l]);
6     while (i + z[i] < sz(s) && s[i + z[i]] == s[z[i]]) z[i]++;
7     if (i + z[i] > r) l = i, r = i + z[i];
8   }
9   return z;
10 }
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
1 array<vi, 2> manacher(const string& s) {
2   int n = sz(s);
3   array<vi,2> p = {vi(n+1), vi(n)};
4   rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
5     int t = r-i+l;
6     if (i<r) p[z][i] = min(t, p[z][l+t]);
7     int L = i-p[z][i], R = i+p[z][i]-l;
8     while (L>=1 && R+1<n && s[L-1] == s[R+1])
9       p[z][i]++, L--, R++;
10    if (R>r) l=L, r=R;
11  }
12  return p;
13 }
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
1 int minRotation(string s) {
2   int a=0, N=sz(s); s += s;
3   rep(b,0,N) rep(k,0,N) {
4     if (a+k == b || s[a+k] < s[b+k]) { b += max(0, k-1); break; }
5     if (s[a+k] > s[b+k]) { a = b; break; }
6   }
7   return a;
8 }
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n + 1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any zero bytes.

Time: $\mathcal{O}(n \log n)$

0579a1, 21 lines

```
1 struct SA {
2   vi sa, lcp;
3   SA(string& s, int lim=256) { // or basic_string<int>
4     int n = sz(s)+1, k = 0, a, b;
5     vi x(all(s)), y(n), ws(max(n, lim));
6     x.push_back(0), sa = lcp = y, iota(all(sa), 0);
7     for(int j = 0, p = 0; p < n; j = max(1, j*2), lim = p) {
8       p = j, iota(all(y), n-j);
9       rep(i, 0, n) if(sa[i] >= j) y[p++] = sa[i]-j;
10      fill(all(ws), 0);
11      rep(i, 0, n) ws[x[i]]++;
12      rep(i, 1, lim) ws[i] += ws[i-1];
13      for(int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
14      swap(x, y), p = 1, x[sa[0]] = 0;
15      rep(i, 1, n) a = sa[i-1], b = sa[i], x[b] =
16        (y[a] == y[b] && y[a+j] == y[b+j]) ? p-1 : p++;
17    }
18    for(int i = 0, j; i < n-1; lcp[x[i++]] = k)
19      for(k && k--, j = sa[x[i]-1]; s[i+k] == s[j+k]; k++);
20  }
21};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r] into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r] substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol - otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$

aae0b8, 44 lines

```
1 struct SuffixTree {
2   enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
3   int toi(char c) { return c - 'a'; }
4   string a; // v = cur node, q = cur position
5   int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;
6   void ukkadd(int i, int c) { suff:
7     if (r[v]<=q) {
8       if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
9         p[m++]=v; v=s[v]; q=r[v]; goto suff; }
10      v=t[v][c]; q=l[v];
11    }
12    if (q==-1 || c==toi(a[q])) q++; else {
13      l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
14      p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
15      l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
16      v=s[p[m]]; q=l[m];
17      while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
18      if (q==r[m]) s[m]=v; else s[m]=m+2;
19      q=r[v]-(q-r[m]); m+=2; goto suff;
20    }
21  }
22  SuffixTree(string a) : a(a) {
23    fill(r,r+N,sz(a)); memset(s, 0, sizeof s);
24    memset(t, -1, sizeof t); fill(t[1],t[1]+ALPHA,0);
25    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
26    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
27  }
28  // example: find longest common substring (uses ALPHA = 28)
29  pii best;
30  int lcs(int node, int i1, int i2, int olen) {
31    if (l[node] <= i1 && i1 < r[node]) return 1;
32    if (l[node] <= i2 && i2 < r[node]) return 2;
33    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
34    rep(c,0,ALPHA) if (t[node][c] != -1)
35      mask |= lcs(t[node][c], i1, i2, len);
36    if (mask == 3) best = max(best, {len, r[node] - len});
37    return mask;
38  }
```

```
39 static pii LCS(string s, string t) {
40   SuffixTree st(s + (char)('z'+1) + t + (char)('z'+2));
41   st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
42   return st.best;
43 }
44};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(—, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

f35677, 58 lines

```
1 struct AhoCorasick {
2   enum {alpha = 26, first = 'A'}; // change this!
3   struct Node {
4     // (nmatches is optional)
5     int back, next[alpha], start = -1, end = -1, nmatches = 0;
6     Node(int v) { memset(next, v, sizeof(next)); }
7   };
8   vector<Node> N; vi backp;
9   void insert(string& s, int j) {
10    assert(!s.empty());
11    int n = 0;
12    for (char c : s) {
13      int& m = N[n].next[c - first];
14      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
15      else n = m;
16    }
17    if (N[n].end == -1) N[n].start = j;
18    backp.push_back(N[n].end);
19    N[n].end = j; N[n].nmatches++;
20  }
21  AhoCorasick(vector<string>& pat) : N(1, -1) {
22    rep(i,0,sz(pat)) insert(pat[i], i);
23    N[0].back = sz(N); N.emplace_back(0);
24    queue<int> q;
25    for (q.push(0); !q.empty(); q.pop()) {
26      int n = q.front(), prev = N[n].back;
27      rep(i,0,alpha) {
28        int &ed = N[n].next[i], y = N[prev].next[i];
29        if (ed == -1) ed = y;
30        else {
31          N[ed].back = y;
32          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
33            = N[y].end;
34          N[ed].nmatches += N[y].nmatches;
35          q.push(ed);
36        } } }
37  }
38  vi find(string word) {
39    int n = 0;
40    vi res; // ll count = 0;
41    for (char c : word) {
42      n = N[n].next[c - first];
43      res.push_back(N[n].end);
44      // count += N[n].nmatches;
45    }
46    return res;
47  }
48  vector<vi> findAll(vector<string>& pat, string word) {
49    vi r = find(word); vector<vi> res(sz(word));
50    rep(i,0,sz(word)) {
51      int ind = r[i];
52      while (ind != -1) {
53        res[i - sz(pat[ind]) + 1].push_back(ind);
54        ind = backp[ind];
55      }
56      return res;
57    }
58};
```

AhoCorasick.py

Description: None

31 lines

```
1 trie = [[0] * 26]
2 def insert(s):
3     cur = 0
4     for c in s:
5         x = ord(c) - ord("A")
6         if not trie[cur][x]:
7             trie[cur][x] = len(trie)
8             trie.append([0] * 26)
9         cur = trie[cur][x]
10    return cur
11 # insert all the strings here first
12 queue = [0]
13 fail = [0] * len(trie)
14 fail[0] = -1
15 for i in range(len(trie)):
16     c = queue[i]
17     for x in range(26):
18         if not trie[c][x]:
19             continue
20         p = fail[c]
21         while p > -1:
22             if trie[p][x]:
23                 fail[trie[c][x]] = trie[p][x]
24                 break
25         p = fail[p]
26     queue.append(trie[c][x])
27 fail[0] = 0
28 go = [[0] * 26 for _ in range(len(trie))]
29 for c in queue:
30     for x in range(26):
31         go[c][x] = trie[c][x] if trie[c][x] else go[fail[c]][x]
```


Math (8)

8.1 Lucas's theorem

For non-negative integers *m* and *n* and a prime *p*, the following congruence relation holds:

$$\binom{m}{n} \equiv \prod_{i=0}^k \binom{m_i}{n_i} \pmod{p},$$

where

$$m = m_kp^k + m_{k-1}p^{k-1} + \cdots + m_1p + m_0,$$

and

$$n = n_kp^k + n_{k-1}p^{k-1} + \cdots + n_1p + n_0$$

are the base *p* expansions of *m* and *n* respectively. This uses the convention that

(

m
n

)

=
0

 if *m* < *n*.

8.2 Group theory

8.2.1 Pólya enumeration theorem

The enumeration theorem employs a multivariate generating function called the cycle index:

$$Z_G(t_1,t_2,\ldots,t_n)=\frac{1}{|G|}\sum_{g\in G}t_1^{j_1(g)}t_2^{j_2(g)}\ldots t_n^{j_n(g)},$$

where *n* is the number of elements of *X* and *j_k(g)* is the number of *k*-cycles of the group element *g* as a permutation of *X*.

The theorem states that the generating function F of the number of colored arrangements by weight is given by:

$$F(t)=Z_G(f(t),f(t^2),f(t^3),\ldots,f(t^n)),$$

or in the multivariate case:

$$F(t_1,\ldots)=Z_G(f(t_1,\ldots),f(t_1^2,\ldots),f(t_1^3,\ldots),\ldots,f(t_1^n,\ldots)).$$

For instance, when seperating the graphs with the number of edges, we let *f(t)* = 1 + *t*, and examine the coefficient of *tⁱ* for a graph with *i* edges, and when seperating the necklaces with the number of beads with three different colors, we let *f(x,y,z)* = *x* + *y* + *z*, and examine the coefficient of *xⁱy^jz^k*.

8.3 Game theory

8.3.1 Nim

For simplicity, we denote *a_i* as the number of stones in the *i*-th pile, *M_i(S)* as removing stones with the amount chosen in the set *S* from the *i*-th pile, and *M_i* = *M_i*[1, *a_i*]. Without further explanation, it is assumed that the SG function of a game *SG* = ⊕_{*i*=1}^{*n*} *SG(a_i)*.

M = ⋁_{*i*=1}^{*n*} *M_i*.

Normal: *SG(n)* = *n*.

Misere: The same, opposite if all piles are 1’s.

8.3.2 Nim (powers)

Given *k*, *M* = ⋁_{*i*=1}^{*n*} *M_i*{*k^m*|*m* ≥ 0}.

Normal: If *k* is odd, *SG(n)* = *n*%2. Otherwise,

$$SG(n)=\left\{\begin{array}{ll}2&n\% (k+1)\%2\\n\% (k+1)&{\rm otherwise}.\end{array}\right.$$

8.3.3 Nim (no greater than half)

M = ⋁_{*i*=1}^{*n*} *M_i*[1,

⌈

a

i

2

⌋

.

.

Normal: *SG(2n)* = *n*, *SG(2n + 1)* = *SG(n)*.

8.3.4 Nim (always greater than half)

M = ⋁_{*i*=1}^{*n*} *M_i*[

⌈

a

i

2

⌋

,

a

i

].

Normal: *SG*(0) = 0, *SG(n)* = ⌊log₂ *n*⌋ + 1.

8.3.5 Nim (proper divisors)

M = ⋁_{*i*=1}^{*n*} *M_i*{*x*|*x* > 1 ∧ *a_i*%*x* = 0}.

Normal: *SG*(1) = 0, *SG(n)* = max_{*x*} (*n*%2^{*x*} = 0).

8.3.6 Nim (divisors)

M = ⋁_{*i*=1}^{*n*} *M_i*{*x*|*a_i*%*x* = 0}.

Normal: *SG*(0) = 0, *SG(n)* = 1 + max_{*x*} (*n*%2^{*x*} = 0).

8.3.7 Nim (fixed)

Given a finite set *S*, *M* = ⋁_{*i*=1}^{*n*} *M_i*(*S*).

Normal: *SG₁*(*n*) is eventually periodic.

Given a finite set *S*, *M* = ⋁_{*i*=1}^{*n*} *M_i*(*S* ∪ *a_i*).

Normal: *SG₂*(*n*) = *SG₁*(*n*) + 1.

8.3.8 Moore's Nim

Given *k*, *M* = ⋃{*M_{x₁}* × *M_{x₂}* ··· × *M_{x_l}*|*l* ≤ *k* ∧ ∀*i*(*x_i* < *x_{i+1}*)}.

Normal: Sum all (*a_i*)₂ in base *k* + 1 without carry. Lose if the result is 0.

Misere: The same, except if all piles are 1’s.

8.3.9 Staircase Nim

One can take any number of objects from *a_{i+1}* to *a_i* (*i* ≥ 0).

Normal: Lose if ⊕_{*i*=0}^{(*n*−1)/2} *a_{2*i*+1}* = 0.

8.3.10 Lasker's Nim

M = ⋁_{*i*=1}^{*n*} *M_i* ∪ *S_i*. (*S_i*: Split a pile into two non-empty piles.)

$$\mathrm{Normal:}\;SG(n)=\left\{\begin{array}{ll}n&n\%4=1,2\\n+1&n\%4=3\\n-1&n\%4=0.\end{array}\right.$$

8.3.11 Kayles

M = ⋁_{*i*=1}^{*n*} *M_i*[1, 2] ∪ *MS_i*[1, 2]. (*MS_i*: Split a pile into two non-empty piles after removing stones.)

Normal: Periodic from the 72-th item with period length 12.

8.3.12 Dawson's chess

n stones in a line. One can take a stone if its neighbours are not taken.

Normal: Periodic from the 52-th item with period length 34.

8.3.13 Ferguson game

Two boxes with *m* stones and *n* stones. One can empty any one box and move any positive number of stones from another box to this box each step.

Normal: Lose if both *m* and *n* are odd.

8.3.14 Fibonacci game

n stones. The first player may take any positive number of stones during the first move, but not all of them. After that, each player may take any positive number of stones, but less than twice the number of stones taken during the last turn.

Normal: Win if *n* is not a fibonacci number.

8.3.15 Wythoff's game

Two piles of stones. Players take turns removing stones from one or both piles; when removing stones from both piles, the numbers of stones removed from each pile must be equal.

Normal: Lose if ⌊

√
5
+
1

2

|

A
−
B

|

⌋
=
min
(
A
,
B
)

8.3.16 Mock turtles

n coins in a line. One can turn over any 1, 2, or 3 coins, but the rightmost coin turned must be from head to tail.

Normal: *SG(n)* = 2*n* + [popcount(*n*) is even].

8.3.17 Ruler

n coins in a line. One can turn over any consecutive coins, but the rightmost coin turned must be from head to tail.

Normal: *SG(n)* = lowbit(*n*).

8.3.18 Hackenbush

The game starts with the players drawing a ground line (conventionally, but not necessarily, a horizontal line at the bottom of the paper or other playing area) and several line segments such that each line segment is connected to the ground, either directly at an endpoint, or indirectly, via a chain of other segments connected by endpoints. Any number of segments may meet at a point and thus there may be multiple paths to ground.

On his turn, a player cuts (erases) any line segment of his choice. Every line segment no longer connected to the ground by any path falls (i.e., gets erased). According to the normal play convention of combinatorial game theory, the first player who is unable to move loses.

Played exclusively with vertical stacks of line segments, also referred to as bamboo stalks, the game directly becomes Nim and can be directly analyzed as such. Divergent segments, or trees, add an additional wrinkle to the game and require use of the colon principle stating that when branches come together at a vertex, one may replace the branches by a non-branching stalk of length equal to their nim sum. This principle changes the representation of the game to the more basic version of the bamboo stalks. The last possible set of graphs that can be made are convergent ones, also known as arbitrarily rooted graphs. By using the fusion principle, we can state that all vertices on any cycle may be fused together without changing the value of the graph. Therefore, any convergent graph can also be interpreted as a simple bamboo stalk graph. By combining all three types of graphs we can add complexity to the game, without ever changing the Nim sum of the game, thereby allowing the game to take the strategies of Nim.

8.3.19 Joseph cycle

n players are numbered with 0, 1, 2, ..., *n* − 1. *f_{1,m}* = 0, *f_{n,m}* = (*f_{n−1,m}* + *m*) mod *n*.

8.3.20 Some highly composite numbers

120: 16, 1260: 36, 10080: 72, 110880: 144, 1081080: 256, 1102701600: 1440

8.4 Probability theory

Let *X* be a discrete random variable with probability *p_X(x)* of assuming the value *x*. It will then have an expected value (mean)

μ
=

E

(
X
)
=

∑

x

x

p

X

(
x
)

 and variance

σ

2

=
V
(
X
)
=

E

(

X

2

)
−
(

E

(
X
)

)

2

=

∑

x

(
x
−

E

(
X
)

)

2

p

X

(
x
)

 where

σ
 is the standard deviation. If *X* is instead continuous it will have a probability density function *f_X(x)* and the sums above will instead be integrals with *p_X(x)* replaced by *f_X(x)*. Expectation is linear:

$$\mathbb{E}(aX+bY)=a\mathbb{E}(X)+b\mathbb{E}(Y)$$

For independent *X* and *Y*,

$$V(aX+bY)=a^2V(X)+b^2V(Y).$$

8.4.1 Discrete distributions

8.4.2 Binomial distribution

The number of successes in *n* independent yes/no experiments, each which yields success with probability *p* is Bin(*n*, *p*), *n* = 1, 2, ..., , 0 ≤ *p* ≤ 1.

$$p(k)=\binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu=np,\;\sigma^2=np(1-p)$$

Bin(*n*, *p*) is approximately Po(*np*) for small *p*.

8.4.3 First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability *p* is Fs(*p*), 0 ≤ *p* ≤ 1.

$$p(k)=p(1-p)^{k-1},\;k=1,2,\ldots$$

$$\mu=\frac{1}{p},\;\sigma^2=\frac{1-p}{p^2}$$

8.4.4 Poisson distribution

The number of events occurring in a fixed period of time *t* if these events occur with a known average rate

κ
 and independently of the time since the last event is Po(

λ
),

λ
=
t
κ
.

$$p(k)=e^{-\lambda}\frac{\lambda^k}{k!},\;k=0,1,2,\ldots$$

$$\mu=\lambda,\;\sigma^2=\lambda$$

8.4.5 Continuous distributions

8.4.6 Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$
$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

8.4.7 Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

8.4.8 Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then $aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$

8.5 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution. is a stationary distribution if $\pi = \mathbf{P} \pi$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j/π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree. A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$. A Markov chain is an A-chain if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

8.6 Formulas

8.6.1 Binomial coefficients

$$\binom{n}{k} = (-1)^k \binom{k-n-1}{k}, \sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$$
$$\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}$$
$$\sqrt{1+z} = 1 + \sum_{k=1}^{\infty} \frac{(-1)^{k-1}}{k \times 2^{2k-1}} \binom{2k-2}{k-1} z^k$$
$$\sum_{k=0}^r \binom{r-k}{m} \binom{s+k}{n} = \binom{r+s+1}{m+n+1}$$
$$C_{n,m} = \binom{n+m}{m} - \binom{n+m}{m-1}, n \geq m$$

$$\binom{n}{k} \equiv [n \& k = k] \pmod{2}$$

$$\binom{n_1 + \dots + n_p}{m} = \sum_{k_1 + \dots + k_p = m} \binom{n_1}{k_1} \dots \binom{n_p}{k_p}$$

8.6.2 Fibonacci numbers

$$F(z) = \frac{z}{1 - z - z^2}$$
$$f_n = \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}}, \phi = \frac{1 + \sqrt{5}}{2}, \hat{\phi} = \frac{1 - \sqrt{5}}{2}$$
$$\sum_{k=1}^n f_k = f_{n+2} - 1, \sum_{k=1}^n f_k^2 = f_n f_{n+1}$$
$$\sum_{k=0}^n f_k f_{n-k} = \frac{1}{5} (n-1) f_n + \frac{2}{5} n f_{n-1}$$
$$\frac{f_{2n}}{f_n} = f_{n-1} + f_{n+1}$$
$$f_1 + 2f_2 + 3f_3 + \dots + n f_n = n f_{n+2} - f_{n+3} + 2]$$
$$\gcd(f_m, f_n) = f_{\gcd(m, n)}$$
$$f_n^2 + (-1)^n = f_{n+1} f_{n-1}$$
$$f_{n+k} = f_n f_{k+1} + f_{n-1} f_k$$
$$f_{2n+1} = f_n^2 + f_{n+1}^2$$
$$(-1)^k f_{n-k} = f_n f_{k-1} - f_{n-1} f_k$$
$$\text{Modulo } f_n, f_{m+n+r} \equiv \begin{cases} f_r, & m \bmod 4 = 0; \\ (-1)^{r+1} f_{n-r}, & m \bmod 4 = 1; \\ (-1)^n f_r, & m \bmod 4 = 2; \\ (-1)^{r+1+n} f_{n-r}, & m \bmod 4 = 3. \end{cases}$$

Period modulo a prime p is a factor of $2p + 2$ or $p - 1$. Only exception: $G(5) = 20$. Period modulo the power of a prime p^k : $G(p^k) = G(p)p^{k-1}$. Period modulo $n = p_1^{k_1} \dots p_m^{k_m}$: $G(n) = \text{lcm}(G(p_1^{k_1}), \dots, G(p_m^{k_m}))$.

8.6.3 Lucas numbers

$$L_0 = 2, L_1 = 1, L_n = L_{n-1} + L_{n-2} = (\frac{1+\sqrt{5}}{2})^n + (\frac{1-\sqrt{5}}{2})^n$$
$$L(x) = \frac{2-x}{1-x-x^2}$$

8.6.4 Catalan numbers

$$c_1 = 1, c_n = \sum_{i=0}^{n-1} c_i c_{n-1-i} = c_{n-1} \frac{4n-2}{n+1} = \frac{\binom{2n}{n}}{n+1}$$
$$= \binom{2n}{n} - \binom{2n}{n-1}, c(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

8.6.5 Stirling cycle numbers

Divide n elements into k non-empty cycles. $s(n, 0) = 0, s(n, n) = 1, s(n+1, k) = s(n, k-1) - ns(n, k)$

$$s(n, k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}$$
$$\begin{bmatrix} n+1 \\ k \end{bmatrix} = n \begin{bmatrix} n \\ k \end{bmatrix} + \begin{bmatrix} n \\ k-1 \end{bmatrix}, \begin{bmatrix} n+1 \\ 2 \end{bmatrix} = n! H_n$$
$$x^{\underline{n}} = x(x-1) \dots (x-n+1) = \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} (-1)^{n-k} x^k$$
$$x^{\overline{n}} = x(x+1) \dots (x+n-1) = \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

8.6.6 Stirling subset numbers

Divide n elements into k non-empty subsets.

$$\begin{Bmatrix} n+1 \\ k \end{Bmatrix} = k \begin{Bmatrix} n \\ k \end{Bmatrix} + \begin{Bmatrix} n \\ k-1 \end{Bmatrix}$$
$$x^n = \sum_{k=0}^n \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\underline{k}} = \sum_{k=0}^n \begin{Bmatrix} n \\ k \end{Bmatrix} (-1)^{n-k} x^{\overline{k}}$$
$$m! \begin{Bmatrix} n \\ m \end{Bmatrix} = \sum_{k=0}^m \binom{m}{k} k^n (-1)^{m-k}$$
$$\sum_{k=1}^n k^p = \sum_{k=0}^p \begin{Bmatrix} p \\ k \end{Bmatrix} (n+1)^{\underline{k}}$$

For a fixed k , generating functions:

$$\sum_{n=0}^{\infty} \begin{Bmatrix} n \\ k \end{Bmatrix} x^{n-k} = \prod_{r=1}^k \frac{1}{1-rx}$$

8.6.7 Motzkin numbers

Draw non-intersecting chords between n points on a circle. Pick n numbers $k_1, k_2, \dots, k_n \in \{-1, 0, 1\}$ so that $\sum_i^a k_i (1 \leq a \leq n)$ is non-negative and the sum of all numbers is 0.

$$M_{n+1} = M_n + \sum_i^{n-1} M_i M_{n-1-i} = \frac{(2n+3)M_n + 3nM_{n-1}}{n+3}$$
$$M_n = \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \binom{n}{2k} \text{Catlan}(k)$$
$$M(X) = \frac{1-x-\sqrt{1-2x-3x^2}}{2x^2}$$

8.6.8 Eulerian numbers

Permutations of the numbers 1 to n in which exactly k elements are greater than the previous element.

$$\left\langle n \atop k \right\rangle = (k+1) \left\langle n-1 \atop k \right\rangle + (n-k) \left\langle n-1 \atop k-1 \right\rangle$$
$$x^n = \sum_k \left\langle n \atop k \right\rangle \binom{x+k}{n}$$
$$\left\langle n \atop m \right\rangle = \sum_{k=0}^m \binom{n+1}{k} (m+1-k)^n (-1)^k$$

8.6.9 Harmonic numbers

Sum of the reciprocals of the first n natural numbers.

$$\sum_{k=1}^n H_k = (n+1)H_n - n$$

$$\sum_{k=1}^n k H_k = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}$$
$$\sum_{k=1}^n \binom{k}{m} H_k = \binom{n+1}{m+1} (H_{n+1} - \frac{1}{m+1})$$

8.6.10 Pentagonal number theorem

$$\prod_{n=1}^{\infty} (1-x^n) = \sum_{n=-\infty}^{\infty} (-1)^k x^{k(3k-1)/2}$$
$$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + \dots$$
$$f(n, k) = p(n) - p(n-k) - p(n-2k) + p(n-5k) + p(n-7k) - \dots$$

8.6.11 Bell numbers

Divide a set that has exactly n elements.

$$B_n = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\}, \quad B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$
$$B_{p^{m+n}} \equiv mB_n + B_{n+1} \pmod{p}$$
$$B(x) = \sum_{n=0}^{\infty} \frac{B_n}{n!} x^n = e^{e^x - 1}$$

8.6.12 Bernoulli numbers

$$B_n = 1 - \sum_{k=0}^{n-1} \binom{n}{k} \frac{B_k}{n-k+1}$$
$$G(x) = \sum_{k=0}^{\infty} \frac{B_k}{k!} x^k = \frac{1}{\sum_{k=0}^{\infty} \frac{x^k}{(k+1)!}}$$
$$\sum_{k=1}^n k^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k n^{m-k+1}$$

8.6.13 Sum of powers

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2$$
$$\sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$
$$\sum_{i=1}^n i^5 = \frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

8.6.14 Sum of squares

Denote $r_k(n)$ the ways to form n with k squares. If:

$$n = 2^{a_0} p_1^{2a_1} \cdots p_r^{2a_r} q_1 b_1 \cdots q_s b_s$$

where $p_i \equiv 3 \pmod{4}$, $q_i \equiv 1 \pmod{4}$, then

$$r_2(n) = \begin{cases} \text{if any } a_i \text{ is a half-integer} \\ 4 \prod_1^r (b_i + 1) & \text{if all } a_i \text{ are integers} \end{cases}$$

$r_3(n) > 0$ when and only when n is not $4^a(8b+7)$.

8.6.15 Derangement

$$D_1 = 0, D_2 = 1, D_n = n! \left(\frac{1}{0!} - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + \frac{(-1)^n}{n!} \right)$$
$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

8.6.16 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \cdots (d_n-1)!)$

8.6.17 Tetrahedron volume

If U, V, W, u, v, w are lengths of edges of the tetrahedron (first three form a triangle; u opposite to U and so on)

$$V = \frac{\sqrt{4u^2v^2w^2 - \sum_{c \in \{y,z\}} u^2(v^2 + w^2 - U^2)^2 + \prod_{c \in \{y,z\}} (v^2 + w^2 - U^2)}}{12}$$

8.7 Integrals

$$\int_L f(x, y, z) ds = \int_{\alpha}^{\beta} f(x(t), y(t), z(t)) \sqrt{x'^2(t) + y'^2(t) + z'^2(t)} dt$$
$$\iint_{\Sigma} f(x, y, z) dS = \iint_D f(x(u, v), y(u, v), z(u, v)) \sqrt{EG - F^2} du dv,$$

where $E = x_u^2 + y_u^2 + z_u^2$, $F = x_u x_v + y_u y_v + z_u z_v$, $G = x_v^2 + y_v^2 + z_v^2$.

$$\int_L P(x, y, z) dx + Q(x, y, z) dy + R(x, y, z) dz$$
$$= \int_a^b [P(x(t), y(t), z(t))x'(t) + Q(x(t), y(t), z(t))y'(t) + R(x(t), y(t), z(t))z'(t)] dt$$

$$\iint_L P(x, y, z) dy dz + Q(x, y, z) dz dx + R(x, y, z) dx dy$$
$$= \pm \iint_D [P(x(u, v), y(u, v), z(u, v)) \frac{\partial(y, z)}{\partial(u, v)} + Q(x(u, v), y(u, v), z(u, v)) \frac{\partial(z, x)}{\partial(u, v)} + R(x(u, v), y(u, v), z(u, v)) \frac{\partial(x, y)}{\partial(u, v)}] du dv$$

8.7.1 Variable substitution

$$\iint_{T(D)} f(x, y) dx dy = \iint_D f(x(u, v), y(u, v)) \left| \frac{\partial(x, y)}{\partial(u, v)} \right| du dv$$

8.7.2 Substitution with polar coordinates

$$x = r \cos \theta, y = r \sin \theta$$
$$\left| \frac{\partial(x, y)}{\partial(r, \theta)} \right| = r$$

8.7.3 Substitution with cylindrical coordinates

$$x = r \cos \theta, y = r \sin \theta, z = z$$
$$\left| \frac{\partial(x, y, z)}{\partial(r, \theta, z)} \right| = r$$

8.7.4 Substitution with spherical coordinates

$$x = r \sin \varphi \cos \theta, y = r \sin \varphi \sin \theta, z = r \cos \varphi$$
$$\left| \frac{\partial(x, y, z)}{\partial(r, \varphi, \theta)} \right| = r^2 \sin \varphi$$

8.7.5 Differentiation

$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$	$(\operatorname{arcsec} x)' = \frac{1}{x\sqrt{1-x^2}}$
$(a^x)' = (\ln a)a^x$	$(\tanh x)' = \operatorname{sech}^2 x$
$(\tan x)' = \sec^2 x$	$(\coth x)' = -\operatorname{csch}^2 x$
$(\cot x)' = \operatorname{csc}^2 x$	$(\operatorname{sech} x)' = -\operatorname{sech} x \tanh x$
$(\sec x)' = \tan x \sec x$	$(\operatorname{csch} x)' = -\operatorname{csch} x \coth x$
$(\csc x)' = -\cot x \csc x$	$(\operatorname{arcsinh} x)' = \frac{1}{\sqrt{1+x^2}}$
$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$	$(\operatorname{arcosh} x)' = \frac{1}{\sqrt{x^2-1}}$
$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$	$(\operatorname{artanh} x)' = \frac{1}{1-x^2}$
$(\arctan x)' = \frac{1}{1+x^2}$	$(\operatorname{arcoth} x)' = \frac{1}{x^2-1}$
$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$	$(\operatorname{arccsch} x)' = -\frac{1}{x^2\sqrt{1+x^2}}$
$(\operatorname{arccsc} x)' = -\frac{1}{x\sqrt{1-x^2}}$	$(\operatorname{arcsech} x)' = -\frac{1}{x\sqrt{1-x^2}}$

8.7.6 Integration

8.7.7 $ax + b$ ($a \neq 0$)

- $\int \frac{x}{ax+b} dx = \frac{1}{a^2} (ax + b - b \ln|ax + b|) + C$
- $\int \frac{x^2}{ax+b} dx = \frac{1}{a^3} \left(\frac{1}{2} (ax + b)^2 - 2b(ax + b) + b^2 \ln|ax + b| \right) + C$
- $\int \frac{dx}{x(ax+b)} = -\frac{1}{b} \ln \left| \frac{ax+b}{x} \right| + C$
- $\int \frac{dx}{x^2(ax+b)} = -\frac{1}{bx} + \frac{a}{b^2} \ln \left| \frac{ax+b}{x} \right| + C$

- $\int \frac{x}{(a+x)b^2} dx = \frac{1}{a^2} (\ln|ax + b| + \frac{b}{ax+b}) + C$
- $\int \frac{x^2}{(a+x)b^2} dx = \frac{1}{a^3} \left(ax + b - 2b \ln|ax + b| - \frac{b^2}{ax+b} \right) + C$
- $\int \frac{dx}{x(ax+b)^2} = \frac{1}{b(ax+b)} - \frac{1}{b^2} \ln \left| \frac{ax+b}{x} \right| + C$

8.7.8 $\sqrt{ax + b}$

- $\int \sqrt{ax + b} dx = \frac{2}{3a} \sqrt{(ax + b)^3} + C$
- $\int x \sqrt{ax + b} dx = \frac{2}{15a^2} (3ax - 2b) \sqrt{(ax + b)^3} + C$
- $\int x^2 \sqrt{ax + b} dx = \frac{2}{105a^3} (15a^2 x^2 - 12abx + 8b^2) \sqrt{(ax + b)^3} + C$
- $\int \frac{x}{\sqrt{ax+b}} dx = \frac{2}{3a^2} (ax - 2b) \sqrt{ax + b} + C$
- $\int \frac{x^2}{\sqrt{ax+b}} dx = \frac{2}{15a^3} (3a^2 x^2 - 4abx + 8b^2) \sqrt{ax + b} + C$
- $\int \frac{dx}{x \sqrt{ax+b}} = \begin{cases} \frac{1}{\sqrt{b}} \ln \left| \frac{\sqrt{ax+b} - \sqrt{b}}{\sqrt{ax+b} + \sqrt{b}} \right| + C & (b > 0) \\ \frac{2}{\sqrt{-b}} \arctan \sqrt{\frac{ax+b}{-b}} + C & (b < 0) \end{cases}$
- $\int \frac{dx}{x^2 \sqrt{ax+b}} = -\frac{\sqrt{ax+b}}{bx} - \frac{a}{2b} \int \frac{dx}{x \sqrt{ax+b}}$
- $\int \frac{\sqrt{ax+b}}{x} dx = -\frac{\sqrt{ax+b}}{x} + \frac{a}{2} \int \frac{dx}{x \sqrt{ax+b}}$

8.7.9 $x^2 \pm a^2$

- $\int \frac{dx}{x^2+a^2} = \frac{1}{a} \arctan \frac{x}{a} + C$
- $\int \frac{dx}{(x^2+a^2)^n} = \frac{x}{2(n-1)a^2(x^2+a^2)^{n-1}} + \frac{2n-3}{2(n-1)a^2} \int \frac{dx}{(x^2+a^2)^{n-1}}$
- $\int \frac{dx}{x^2-a^2} = \frac{1}{2a} \ln \left| \frac{x-a}{x+a} \right| + C$

8.7.10 $ax^2 + b$ ($a > 0$)

- $\int \frac{dx}{ax^2+b} = \begin{cases} \frac{1}{\sqrt{ab}} \arctan \sqrt{\frac{a}{b}} x + C & (b > 0) \\ \frac{1}{2\sqrt{-ab}} \ln \left| \frac{\sqrt{ax-b}}{\sqrt{ax+b}} \right| + C & (b < 0) \end{cases}$
- $\int \frac{x}{ax^2+b} dx = \frac{1}{2a} \ln|ax^2 + b| + C$
- $\int \frac{x^2}{ax^2+b} dx = \frac{x}{a} - \frac{b}{a} \int \frac{dx}{ax^2+b}$
- $\int \frac{dx}{x(ax^2+b)} = \frac{1}{2b} \ln \frac{x^2}{|ax^2+b|} + C$
- $\int \frac{dx}{x^2(ax^2+b)} = -\frac{1}{bx} - \frac{a}{b} \int \frac{dx}{ax^2+b}$
- $\int \frac{dx}{x^3(ax^2+b)} = \frac{a}{2b^2} \ln \frac{|ax^2+b|}{x^2} - \frac{1}{2bx^2} + C$
- $\int \frac{dx}{(ax^2+b)^2} = \frac{x}{2b(ax^2+b)} + \frac{1}{2b} \int \frac{dx}{ax^2+b}$

8.7.11 $ax^2 + bx + c$ ($a > 0$)

- $\frac{dx}{ax^2+bx+c} = \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan \frac{2ax+b}{\sqrt{4ac-b^2}} + C & (b^2 < 4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln \left| \frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right| + C & (b^2 > 4ac) \end{cases}$
- $\int \frac{x}{ax^2+bx+c} dx = \frac{1}{2a} \ln|ax^2 + bx + c| - \frac{b}{2a} \int \frac{dx}{ax^2+bx+c}$

8.7.12 $\sqrt{x^2 + a^2} \ (a > 0)$

- $\int \frac{dx}{\sqrt{x^2 + a^2}} = \operatorname{arsh} \frac{x}{a} + C_1 = \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int \frac{dx}{\sqrt{(x^2 + a^2)^3}} = \frac{x}{a^2 \sqrt{x^2 + a^2}} + C$
- $\int \frac{x}{\sqrt{x^2 + a^2}} dx = \sqrt{x^2 + a^2} + C$
- $\int \frac{x}{\sqrt{(x^2 + a^2)^3}} dx = -\frac{1}{\sqrt{x^2 + a^2}} + C$
- $\int \frac{x^2}{\sqrt{x^2 + a^2}} dx = \frac{x}{2} \sqrt{x^2 + a^2} - \frac{a^2}{2} \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int \frac{x^2}{\sqrt{(x^2 + a^2)^3}} dx = -\frac{x}{\sqrt{x^2 + a^2}} + \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int \frac{dx}{x\sqrt{x^2 + a^2}} = \frac{1}{a} \ln \frac{\sqrt{x^2 + a^2} - a}{|x|} + C$
- $\int \frac{dx}{x^2 \sqrt{x^2 + a^2}} = -\frac{\sqrt{x^2 + a^2}}{a^2 x} + C$
- $\int \sqrt{x^2 + a^2} dx = \frac{x}{2} \sqrt{x^2 + a^2} + \frac{a^2}{2} \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int \sqrt{(x^2 + a^2)^3} dx = \frac{x}{8} (2x^2 + 5a^2) \sqrt{x^2 + a^2} + \frac{3}{8} a^4 \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int x \sqrt{x^2 + a^2} dx = \frac{1}{3} \sqrt{(x^2 + a^2)^3} + C$
- $\int x^2 \sqrt{x^2 + a^2} dx = \frac{x}{8} (2x^2 + a^2) \sqrt{x^2 + a^2} - \frac{a^4}{8} \ln(x + \sqrt{x^2 + a^2}) + C$
- $\int \frac{\sqrt{x^2 + a^2}}{x} dx = \sqrt{x^2 + a^2} + a \ln \frac{\sqrt{x^2 + a^2} - a}{|x|} + C$
- $\int \frac{\sqrt{x^2 + a^2}}{x^2} dx = -\frac{\sqrt{x^2 + a^2}}{x} + \ln(x + \sqrt{x^2 + a^2}) + C$

8.7.13 $\sqrt{x^2 - a^2} \ (a > 0)$

- $\int \frac{dx}{\sqrt{x^2 - a^2}} = \frac{x}{|x|} \operatorname{arch} \frac{|x|}{a} + C_1 = \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int \frac{dx}{\sqrt{(x^2 - a^2)^3}} = -\frac{x}{a^2 \sqrt{x^2 - a^2}} + C$
- $\int \frac{x}{\sqrt{x^2 - a^2}} dx = \sqrt{x^2 - a^2} + C$
- $\int \frac{x}{\sqrt{(x^2 - a^2)^3}} dx = -\frac{1}{\sqrt{x^2 - a^2}} + C$
- $\int \frac{x^2}{\sqrt{x^2 - a^2}} dx = \frac{x}{2} \sqrt{x^2 - a^2} + \frac{a^2}{2} \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int \frac{x^2}{\sqrt{(x^2 - a^2)^3}} dx = -\frac{x}{\sqrt{x^2 - a^2}} + \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int \frac{dx}{x\sqrt{x^2 - a^2}} = \frac{1}{a} \operatorname{arccos} \frac{a}{|x|} + C$
- $\int \frac{dx}{x^2 \sqrt{x^2 - a^2}} = \frac{\sqrt{x^2 - a^2}}{a^2 x} + C$
- $\int \sqrt{x^2 - a^2} dx = \frac{x}{2} \sqrt{x^2 - a^2} - \frac{a^2}{2} \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int \sqrt{(x^2 - a^2)^3} dx = \frac{x}{8} (2x^2 - 5a^2) \sqrt{x^2 - a^2} + \frac{3}{8} a^4 \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int x \sqrt{x^2 - a^2} dx = \frac{1}{3} \sqrt{(x^2 - a^2)^3} + C$
- $\int x^2 \sqrt{x^2 - a^2} dx = \frac{x}{8} (2x^2 - a^2) \sqrt{x^2 - a^2} - \frac{a^4}{8} \ln|x + \sqrt{x^2 - a^2}| + C$
- $\int \frac{\sqrt{x^2 - a^2}}{x} dx = \sqrt{x^2 - a^2} - a \operatorname{arccos} \frac{a}{|x|} + C$
- $\int \frac{\sqrt{x^2 - a^2}}{x^2} dx = -\frac{\sqrt{x^2 - a^2}}{x} + \ln|x + \sqrt{x^2 - a^2}| + C$

8.7.14 $\sqrt{a^2 - x^2} \ (a > 0)$

- $\int \frac{dx}{\sqrt{a^2 - x^2}} = \arcsin \frac{x}{a} + C$
- $\frac{dx}{\sqrt{(a^2 - x^2)^3}} = \frac{x}{a^2 \sqrt{a^2 - x^2}} + C$
- $\int \frac{x}{\sqrt{a^2 - x^2}} dx = -\sqrt{a^2 - x^2} + C$
- $\int \frac{x}{\sqrt{(a^2 - x^2)^3}} dx = \frac{1}{\sqrt{a^2 - x^2}} + C$

- $\int \frac{x^2}{\sqrt{a^2 - x^2}} dx = -\frac{x}{2} \sqrt{a^2 - x^2} + \frac{a^2}{2} \arcsin \frac{x}{a} + C$
- $\int \frac{x^2}{\sqrt{(a^2 - x^2)^3}} dx = \frac{x}{\sqrt{a^2 - x^2}} - \arcsin \frac{x}{a} + C$
- $\int \frac{dx}{x\sqrt{a^2 - x^2}} = \frac{1}{a} \ln \frac{a - \sqrt{a^2 - x^2}}{|x|} + C$
- $\int \frac{dx}{x^2 \sqrt{a^2 - x^2}} = -\frac{\sqrt{a^2 - x^2}}{a^2 x} + C$
- $\int \sqrt{a^2 - x^2} dx = \frac{x}{2} \sqrt{a^2 - x^2} + \frac{a^2}{2} \arcsin \frac{x}{a} + C$
- $\int \sqrt{(a^2 - x^2)^3} dx = \frac{x}{8} (5a^2 - 2x^2) \sqrt{a^2 - x^2} + \frac{3}{8} a^4 \arcsin \frac{x}{a} + C$
- $\int x \sqrt{a^2 - x^2} dx = -\frac{1}{3} \sqrt{(a^2 - x^2)^3} + C$
- $\int x^2 \sqrt{a^2 - x^2} dx = \frac{x}{8} (2x^2 - a^2) \sqrt{a^2 - x^2} + \frac{a^4}{8} \arcsin \frac{x}{a} + C$
- $\int \frac{\sqrt{a^2 - x^2}}{x} dx = \sqrt{a^2 - x^2} + a \ln \frac{a - \sqrt{a^2 - x^2}}{|x|} + C$
- $\int \frac{\sqrt{a^2 - x^2}}{x^2} dx = -\frac{\sqrt{a^2 - x^2}}{x} - \arcsin \frac{x}{a} + C$

8.7.15 $\sqrt{\pm ax^2 + bx + c} \ (a > 0)$

- $\int \frac{dx}{\sqrt{ax^2 + bx + c}} = \frac{1}{\sqrt{a}} \ln |2ax + b + 2\sqrt{a\sqrt{ax^2 + bx + c}}| + C$
- $\int \sqrt{ax^2 + bx + c} dx = \frac{2ax + b}{4a} \sqrt{ax^2 + bx + c} + \frac{4ac - b^2}{8\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a\sqrt{ax^2 + bx + c}}| + C$
- $\int \frac{x}{\sqrt{ax^2 + bx + c}} dx = \frac{1}{a} \sqrt{ax^2 + bx + c} - \frac{b}{2\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a\sqrt{ax^2 + bx + c}}| + C$
- $\int \frac{dx}{\sqrt{c + bx - ax^2}} = -\frac{1}{\sqrt{a}} \arcsin \frac{2ax - b}{\sqrt{b^2 + 4ac}} + C$
-

$$\int \sqrt{c + bx - ax^2} dx = \frac{2ax - b}{4a} \sqrt{c + bx - ax^2} +$$

$$\frac{b^2 + 4ac}{8\sqrt{a^3}} \arcsin \frac{2ax - b}{\sqrt{b^2 + 4ac}} + C$$

- $\int \frac{x}{\sqrt{c + bx - ax^2}} dx = -\frac{1}{a} \sqrt{c + bx - ax^2} + \frac{b}{2\sqrt{a^3}} \arcsin \frac{2ax - b}{\sqrt{b^2 + 4ac}} + C$

8.7.16 $\sqrt{\pm \frac{x-a}{x-b}} \ \& \ \sqrt{(x-a)(x-b)}$

- $\int \sqrt{\frac{x-a}{x-b}} dx = (x-b) \sqrt{\frac{x-a}{x-b}} + (b-a) \ln(|\sqrt{x-a}| + |\sqrt{x-b}|) + C$
- $\int \sqrt{\frac{x-a}{b-x}} dx = (x-b) \sqrt{\frac{x-a}{b-x}} + (b-a) \arcsin \sqrt{\frac{x-a}{b-x}} + C$
- $\int \frac{dx}{\sqrt{(x-a)(b-x)}} = 2 \arcsin \sqrt{\frac{x-a}{b-x}} + C \ (a < b)$
- $\int \sqrt{(x-a)(b-x)} dx = \frac{2x-a-b}{4} \sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4} \arcsin \sqrt{\frac{x-a}{b-x}} + C \ (a < b)$

8.7.17 **Triangular function**

- $\int \tan x dx = -\ln |\cos x| + C$
- $\int \cot x dx = \ln |\sin x| + C$
- $\int \sec x dx = \ln \left| \tan \left(\frac{\pi}{4} + \frac{x}{2} \right) \right| + C = \ln |\sec x + \tan x| + C$
- $\int \csc x dx = \ln \left| \tan \frac{x}{2} \right| + C = \ln |\csc x - \cot x| + C$
- $\int \sec^2 x dx = \tan x + C$
- $\int \csc^2 x dx = -\cot x + C$
- $\int \sec x \tan x dx = \sec x + C$
- $\int \csc x \cot x dx = -\csc x + C$
- $\int \sin^2 x dx = \frac{x}{2} - \frac{1}{4} \sin 2x + C$
- $\int \cos^2 x dx = \frac{x}{2} + \frac{1}{4} \sin 2x + C$
- $\int \sin^n x dx = -\frac{1}{n} \sin^{n-1} x \cos x + \frac{n-1}{n} \int \sin^{n-2} x dx$

- $\int \cos^n x dx = \frac{1}{n} \cos^{n-1} x \sin x + \frac{n-1}{n} \int \cos^{n-2} x dx$

$$13. \frac{dx}{\sin^n x} = -\frac{1}{n-1} \frac{\cos x}{\sin^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\sin^{n-2} x}$$

$$14. \frac{dx}{\cos^n x} = \frac{1}{n-1} \frac{\sin x}{\cos^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\cos^{n-2} x}$$

15.

$$\int \cos^m x \sin^n x dx$$

$$= \frac{1}{m+n} \cos^{m-1} x \sin^{n+1} x + \frac{m-1}{m+n} \int \cos^{m-2} x \sin^n x dx$$

$$= -\frac{1}{m+n} \cos^{m+1} x \sin^{n-1} x + \frac{n-1}{m+1} \int \cos^m x \sin^{n-2} x dx$$

$$16. \int \sin ax \cos bx dx = -\frac{1}{2(a+b)} \cos(a+b)x - \frac{1}{2(a-b)} \cos(a-b)x + C$$

$$17. \int \sin ax \sin bx dx = -\frac{1}{2(a+b)} \sin(a+b)x + \frac{1}{2(a-b)} \sin(a-b)x + C$$

$$18. \int \cos ax \cos bx dx = \frac{1}{2(a+b)} \sin(a+b)x + \frac{1}{2(a-b)} \sin(a-b)x + C$$

$$19. \int \frac{dx}{a+b \sin x} = \begin{cases} \frac{2}{\sqrt{a^2 - b^2}} \arctan \frac{a \tan \frac{x}{2} + b}{\sqrt{a^2 - b^2}} + C & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2 - a^2}} \ln \left| \frac{a \tan \frac{x}{2} + b - \sqrt{b^2 - a^2}}{a \tan \frac{x}{2} + b + \sqrt{b^2 - a^2}} \right| + C & (a^2 < b^2) \end{cases}$$

$$20. \int \frac{dx}{a+b \cos x} = \begin{cases} \frac{2}{a+b} \sqrt{\frac{a+b}{a-b}} \arctan \left(\sqrt{\frac{a-b}{a+b}} \tan \frac{x}{2} \right) + C & (a^2 > b^2) \\ \frac{1}{a+b} \sqrt{\frac{a+b}{a-b}} \ln \left| \frac{\tan \frac{x}{2} + \sqrt{\frac{a+b}{a-b}}}{\tan \frac{x}{2} - \sqrt{\frac{a+b}{a-b}}} \right| + C & (a^2 < b^2) \end{cases}$$

$$21. \int \frac{dx}{a^2 \cos^2 x + b^2 \sin^2 x} = \frac{1}{ab} \arctan \left(\frac{b}{a} \tan x \right) + C$$

$$22. \int \frac{dx}{a^2 \cos^2 x - b^2 \sin^2 x} = \frac{1}{2ab} \ln \left| \frac{b \tan x + a}{b \tan x - a} \right| + C$$

$$23. \int x \sin ax dx = \frac{1}{a^2} \sin ax - \frac{1}{a} x \cos ax + C$$

$$24. \int x^2 \sin ax dx = -\frac{1}{a} x^2 \cos ax + \frac{2}{a^2} x \sin ax + \frac{2}{a^3} \cos ax + C$$

$$25. \int x \cos ax dx = \frac{1}{a^2} \cos ax + \frac{1}{a} x \sin ax + C$$

$$26. \int x^2 \cos ax dx = \frac{1}{a} x^2 \sin ax + \frac{2}{a^2} x \cos ax - \frac{2}{a^3} \sin ax + C$$

8.7.18 **Inverse triangular function** $(a > 0)$

- $\int \arcsin \frac{x}{a} dx = x \arcsin \frac{x}{a} + \sqrt{a^2 - x^2} + C$
- $\int x \arcsin \frac{x}{a} dx = \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arcsin \frac{x}{a} + \frac{x}{4} \sqrt{x^2 - a^2} + C$
- $\int x^2 \arcsin \frac{x}{a} dx = \frac{x^3}{3} \arcsin \frac{x}{a} + \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C$
- $\int \arccos \frac{x}{a} dx = x \arccos \frac{x}{a} - \sqrt{a^2 - x^2} + C$
- $\int x \arccos \frac{x}{a} dx = \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arccos \frac{x}{a} - \frac{x}{4} \sqrt{a^2 - x^2} + C$
- $\int x^2 \arccos \frac{x}{a} dx = \frac{x^3}{3} \arccos \frac{x}{a} - \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C$
- $\int \arctan \frac{x}{a} dx = x \arctan \frac{x}{a} - \frac{a}{2} \ln(a^2 + x^2) + C$
- $\int x \arctan \frac{x}{a} dx = \frac{1}{2} (a^2 + x^2) \arctan \frac{x}{a} - \frac{a}{2} x + C$
- $\int x^2 \arctan \frac{x}{a} dx = \frac{x^3}{3} \arctan \frac{x}{a} - \frac{a}{6} x^2 + \frac{a^3}{6} \ln(a^2 + x^2) + C$

8.7.19 **Exponential function**

- $\int a^x dx = \frac{1}{\ln a} a^x + C$
- $\int e^{ax} dx = \frac{1}{a} e^{ax} + C$
- $\int x e^{ax} dx = \frac{1}{a^2} (ax - 1) e^{ax} + C$
- $\int x^n e^{ax} dx = \frac{1}{a} x^n e^{ax} - \frac{n}{a} \int x^{n-1} e^{ax} dx$
- $\int x a^x dx = \frac{x}{\ln a} a^x - \frac{1}{(\ln a)^2} a^x + C$
- $\int x^n a^x dx = \frac{1}{\ln a} x^n a^x - \frac{n}{\ln a} \int x^{n-1} a^x dx$
- $\int e^{ax} \sin bx dx = \frac{1}{a^2 + b^2} e^{ax} (a \sin bx - b \cos bx) + C$
- $\int e^{ax} \cos bx dx = \frac{1}{a^2 + b^2} e^{ax} (b \sin bx + a \cos bx) + C$
- $\int e^{ax} \sin^n bx dx = \frac{1}{a^2 + b^2 n^2} e^{ax} \sin^{n-1} bx (a \sin bx - nb \cos bx) + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \sin^{n-2} bx dx$
- $\int e^{ax} \cos^n bx dx = \frac{1}{a^2 + b^2 n^2} e^{ax} \cos^{n-1} bx (a \cos bx + nb \sin bx) + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \cos^{n-2} bx dx$

8.7.20 Logarithmic function

- 1. $\int \ln x dx = x \ln x - x + C$
- 2. $\int \frac{dx}{x \ln x} = \ln |\ln x| + C$
- 3. $\int x^n \ln x dx = \frac{1}{n+1} x^{n+1} (\ln x - \frac{1}{n+1}) + C$
- 4. $\int (\ln x)^n dx = x (\ln x)^n - n \int (\ln x)^{n-1} dx$
- 5. $\int x^m (\ln x)^n dx = \frac{1}{m+1} x^{m+1} (\ln x)^n - \frac{n}{m+1} \int x^m (\ln x)^{n-1} dx$

8.8 Prufer Code

In combinatorial mathematics, the Prufer sequence of a labeled tree is a unique sequence associated with the tree. The sequence for a tree on n vertices has length $n - 2$.

One can generate a labeled tree's Prufer sequence by iteratively removing vertices from the tree until only two vertices remain. Specifically, consider a labeled tree T with vertices $1, 2, \dots, n$. At step i , remove the leaf with the smallest label and set the i th element of the Prufer sequence to be the label of this leaf's neighbor.

One can generate a labeled tree from a sequence in three steps. The tree will have $n + 2$ nodes, numbered from 1 to $n + 2$. For each node set its degree to the number of times it appears in the sequence plus 1. Next, for each number in the sequence $a[i]$, find the first (lowest-numbered) node, j , with degree equal to 1, add the edge $(j, a[i])$ to the tree, and decrement the degrees of j and $a[i]$. At the end of this loop two nodes with degree 1 will remain (call them u, v). Lastly, add the edge (u, v) to the tree.

The Prufer sequence of a labeled tree on n vertices is a unique sequence of length $n - 2$ on the labels 1 to n - this much is clear. Somewhat less obvious is the fact that for a given sequence S of length $n - 2$ on the labels 1 to n , there is a unique labeled tree whose Prufer sequence is S .

Sources (9)

Nearly all content is taken from **KACTL** by KTH's ICPC team at github.com/kth-competitive-programming/kactl and **cp-reference** by Zhongtang Luo (zhtluo) at github.com/zhtluo/cp-reference.